“Año del Bicentenario, de la consolidación de nuestra Independencia, y de la conmemoración de las heroicas batallas de Junín y Ayacucho”

UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ

FACULTAD DE INGENIERÍA DE SISTEMAS



INFORME

Red neuronal con técnicas de machine learning que permita la conducción autónoma de un carrito de juguete

INTEGRANTES:

Ramos Tacza, Camilda

Torres ambrosio Katherine Medally

DOCENTE:

Msc. Jaime Antonio Huaytalla Pariona

ASIGNATURA:

Tecnologías Emergentes

HUANCAYO - PERÚ

2024

Índice:

1. Red neuronal con técnicas de machine learning que permita la conducción autónoma de un carrito de juguete 3
    Explicación del código 3
        1. Importar librerias 3
        2. Recopilación de los datos 4
        3. Procesamiento de Datos 5
        4. División de datos 7
        5. Definición de modelo de red neuronal 8
        6. Compilar el modelo 9
        7. Entrenamiento del modelo 9
        8. Gráfico de pérdida durante el entrenamiento 11
        9. Evaluación del modelo 13
        10. Implementación en el carrito del juguete 14
        11. Ejemplo del uso 15
        12. Graficar las predicciones 17
2. Red neuronal para predecir la diabetes en base a los atributos especificados. 19
    Explicación del código: 20
        1. Importación de librerías 20
        2. Montar Google Drive en Colab 20
        3. Ruta al archivo de datos, carga y visualización de datos 21
        4. Leer datos desde un archivo Excel y preparar los datos 21
        5. Dividir los datos en conjuntos de entrenamiento y prueba e Imprimir los conjuntos de datos 21
        6. Definir el modelo y Compilar el modelo 22
        7. Entrenar el modelo 22
        8. Evaluar el modelo 23
        9. Realizar predicciones 23
        10. Convertir predicciones a binario 23
        11. Graficar las pérdidas: 24
3. Clasificación binaria 25
    Explicacion del codigo: 25
        1. Importar las bibliotecas 25
        2. Carga del dataset 26
        3. División de los datos 27
        4. Entrenamiento del Modelo 27
        5. Predicciones y Evaluación 28
        6. Matriz de confusión 29
        7. Histograma de predicciones 31
        8. Imprimir resultados 32
4. Modelo de clasificación de imágenes utilizando una red neuronal convolucional (CNN). 32

Explicacion del codigo: 33
1. Importación de librerías 33
2. Carga del dataset CIFAR-10 33
3. Normalización de los datos 34
4. Conversión de etiquetas a one-hot encoding 34
5. Construcción del modelo CNN 34
6. Compilación del modelo 36
7. Entrenamiento del modelo 37
8. Evaluación del modelo 37
10. Función para predecir una nueva imagen 39
11. Google Drive 40
12. Prueba con una imagen nueva 40

TECNOLOGÍAS EMERGENTES
INFORME

Link de la carpeta de los códigos: https://drive.google.com/drive/folders/1qLfVz_e73upDaoXqdoJbjEBhD86bshRZ?usp=drive_link

1. Red neuronal con técnicas de machine learning que permita la conducción autónoma de un carrito de juguete

El código proporcionado es sobre cómo entrenar una red neuronal utilizando técnicas de machine learning para permitir la conducción autónoma de un carrito de juguete basado en sensores de distancia y ubicación del obstáculo. La red neuronal aprende a tomar decisiones sobre la dirección y el giro del carrito en función de la lectura del sensor.

Explicación del código

1. Importar librerias

```
[1] import numpy as np #operaciones numericas
    import tensorflow as tf #plataforma de codigo abierto para aprendizaje automatico
    from tensorflow import keras #para construir y entrenar la red neuronal
    from sklearn.model_selection import train_test_split #Divide los datos en conjuntos de entrenamiento y prueba
    import matplotlib.pyplot as plt #Crea visualizaciones estaticas, animadas e interactivas en python
```

Se importan las bibliotecas necesarias:

- **"numpy"** para operaciones numéricas también se utiliza para crear el conjunto de datos del entrenamiento y manipular estos datos
- **"tensorflow"** es una plataforma de código abierto desarrollada por Google para la programación de flujo de datos y el aprendizaje automático a gran escala.
- **"tensorflow"** y **"keras"** se utilizan para definir, compilar y entrenar el modelo de red neuronal que aprenderá a predecir las acciones basada en las lecturas del sensor.
- **"train_test_split"** de **"skelearn.model_selection"** facilita la separación de los datos en subconjuntos para entrenar y evaluar el modelo, lo que ayuda a evitar el sobreajuste y a medir el rendimiento del modelo de manera más precisa.
- **"matplotlib.pyplot"** se utiliza para crear gráficos y visualizar datos. También permite visualizar la pérdida del modelo durante el entrenamiento y la visualización de las predicciones del modelo

2. *Recopilación de los datos*

Se simulan datos de entrenamiento que consisten en lecturas del sensor de distancia y ubicación del obstáculo, junto con las acciones correspondientes (dirección y giro). Estos datos se almacenan en *"datos_entrenamiento"*

En esta parte del código se contienen pares de datos que representan diferentes escenarios para un carrito de juguete con un sensor de distancia y una ubicación de obstáculo, junto con las acciones correspondientes que el carrito debe tomar.

```
[2]  # Recopilación de Datos
     # Se simulan datos de entrenamiento para el sensor de distancia y ubicación del obstáculo, junto con las acciones correspondientes
     datos_entrenamiento = np.array([
         ([0, -1], [1, 1]),   # No hay obstáculo, giro a la derecha
         ([0.5, -1], [1, 1]), # Se acerca a un obstáculo a la izquierda, giro a la derecha
         ([1, -1], [-1, 1]),  # Obstáculo demasiado cerca a la izquierda, giro a la izquierda
         ([0, 1], [1, -1]),   # No hay obstáculo, giro a la izquierda
         ([0.5, 1], [1, -1]), # Se acerca a un obstáculo a la derecha, giro a la izquierda
         ([1, 1], [-1, -1])   # Obstáculo demasiado cerca a la derecha, giro a la izquierda
     ], dtype=float)
```

- *"np.array([...], dtype=float)"* crea un arreglo NumPy a partir de la lista de datos proporcionada. La opción *"dtype=float"* asegura que todos los valores en el arreglo sean tipo *"float"*

El arreglo contiene seis elementos, cada uno de los cuales es una tupla de dos listas:

- La primera lista de cada tupla representa las características de entrada: *"[distancia, ubicación]"*
- La segunda lista de cada tupla representa las acciones de salida: *"[dirección, giro]"*

Características de Entrada [distancia, ubicación]

- Distancia: Representa la distancia entre el carrito y el obstáculo, con valores entre 0 (no hay obstáculo) y 1 (obstáculo muy cercano).
- Ubicación: Indica la posición del obstáculo relativo al carrito, el -1 significa que el obstáculo está a la izquierda y el 1 significa que el obstáculo está a la derecha

Acciones de Salida [dirección, giro]

- dirección: La dirección del movimiento del carrito, 1 indica moverse hacia adelante y -1 indica moverse hacia atrás
- giro: La dirección del giro del carrito, 1 indica un giro a la derecha y -1 indica un giro a la izquierda.

Detalle de Cada Tupla

1. ([0, -1], [1, 1])
    - Entrada: [0, -1] (No hay obstáculo, ubicado a la izquierda).
    - Salida: [1, 1] (Mover hacia adelante y girar a la derecha).
2. ([0.5, -1], [1, 1])

- Entrada: [0.5, -1] (Obstáculo algo cercano a la izquierda).
- Salida: [1, 1] (Mover hacia adelante y girar a la derecha).

3. ([1, -1], [-1, 1])
   - Entrada: [1, -1] (Obstáculo muy cercano a la izquierda).
   - Salida: [-1, 1] (Mover hacia atrás y girar a la derecha).
4. ([0, 1], [1, -1])
   - Entrada: [0, 1] (No hay obstáculo, ubicado a la derecha).
   - Salida: [1, -1] (Mover hacia adelante y girar a la izquierda).
5. ([0.5, 1], [1, -1])
   - Entrada: [0.5, 1] (Obstáculo algo cercano a la derecha).
   - Salida: [1, -1] (Mover hacia adelante y girar a la izquierda).
6. ([1, 1], [-1, -1])
   - Entrada: [1, 1] (Obstáculo muy cercano a la derecha).
   - Salida: [-1, -1] (Mover hacia atrás y girar a la izquierda).

Razón de los valores asignados

- *Seguridad y Evasión de Obstáculos:* Los valores están asignados de manera que el carrito evite colisiones. Si un obstáculo está muy cercano, el carrito gira o retrocede.

- *Dirección del Movimiento:* Las acciones están diseñadas para mover el carrito de manera segura en la dirección opuesta al obstáculo más cercano.
- *Simulaciones Realistas:* Aunque los valores son simulados, imitan situaciones reales en las que un carrito necesita decidir su dirección y giro basándose en lecturas de sensores.

3. Procesamiento de Datos

Se separan las características (lecturas del sensor) y las etiquetas (acciones de dirección y giro) del conjunto de datos de entrenamiento utilizando indexación de matrices de NumPy.

Las características se almacenan en *"X_train"* (características de la entrada es decir la lectura del sensor*)* y las etiquetas en *"Y_train"* (que son las acciones de dirección y giro)

```
[3]  # Procesamiento de Datos
     # Separar características y etiquetas
     X_train = datos_entrenamiento[:, 0]  # Características (sensor de distancia y ubicación del obstáculo)
     y_train = datos_entrenamiento[:, 1]  # Etiquetas (dirección y giro)
```

datos_entrenamiento[:, 0] => características

Cada entrada en `datos_entrenamiento` es una pareja de listas, donde la primera lista representa las características (sensor de distancia y ubicación del obstáculo). Esta expresión utiliza la indexación avanzada de NumPy para seleccionar todas las filas (indicado por `:`) y la primera columna (indicado por `0`) del arreglo `datos_entrenamiento`.

datos_entrenamiento[:, 1] => etiquetas

La segunda lista en cada entrada representa las etiquetas (acciones en términos de dirección y giro). Esta expresión selecciona todas las filas y la segunda columna (indicado por `1`) del arreglo `datos_entrenamiento`.

Ejemplo ilustrativo:
Dado el arreglo datos_entrenamiento

```
Datos de Entrenamiento Completo:
[[[ 0.  -1. ]
  [ 1.   1. ]]

 [[ 0.5 -1. ]
  [ 1.   1. ]]

 [[ 1.  -1. ]
  [-1.   1. ]]

 [[ 0.   1. ]
  [ 1.  -1. ]]

 [[ 0.5  1. ]
  [ 1.  -1. ]]

 [[ 1.   1. ]
  [-1.  -1. ]]]
```

Cuando seleccionamos las características "X_train)" el resultado es:

```
Características (X_train):
[[ 0.  -1. ]
 [ 0.5 -1. ]
 [ 1.  -1. ]
 [ 0.   1. ]
 [ 0.5  1. ]
 [ 1.   1. ]]
```

Y al seleccionar las etiquetas "y_train" el resultado es:

```
Etiquetas (y_train):
[[ 1.  1.]
 [ 1.  1.]
 [-1.  1.]
 [ 1. -1.]
 [ 1. -1.]
 [-1. -1.]]
```

4. *División de datos*

Se dividen los datos de entrenamiento en conjuntos de entrenamiento y prueba utilizando la función *"train_test_split"* de Scikit-learn. Esto se realiza para evaluar el rendimiento del modelo en datos no vistos. Las características y etiquetas de entrenamiento se dividen en *"X_train", "X_test","y_train" y "y_test"*respectivamente.

```
[4]  # División de Datos
     # Dividir los datos en conjuntos de entrenamiento y prueba
     X_train, X_test, y_train, y_test = train_test_split(X_train, y_train, test_size=0.2, random_state=42)
```

- ***"train_test_split"*** esta función de la biblioteca scikit-learn se utiliza para dividir los datos en dos subconjuntos: uno para entrenamiento y otro para pruebas. Esto es fundamental para evaluar el rendimiento del modelo en datos no vistos durante el entrenamiento, lo que ayuda a detectar si el modelo está sobre ajustando (memorizando) los datos de entrenamiento.

- ***"X_train"*** y ***"y_train"*** son los datos de características (entradas) y etiquetas (salidas) que se utilizarán para entrenar el modelo.

- ***"test_size=0.2"*** este argumento indica que el 20% de los datos se reservarán para el conjunto de prueba. En otras palabras, el 80% de los datos se utilizarán para el entrenamiento.
- "***random_state=42***" este parámetro se utiliza para establecer la semilla aleatoria que controla la división de los datos. Al fijar la semilla aleatoria, garantizamos que la división de datos sea reproducible, es decir, obtendremos la misma división cada vez que ejecutemos el código. El número 42 es arbitrario y se utiliza comúnmente en ejemplos de código como una referencia al "número mágico" en la novela "Guía del autoestopista galáctico".

5. *Definición de modelo de red neuronal*

Se define el modelo de red neuronal utilizando la API secuencial de Keras. El modelo consta de dos capas ocultas, cada una con 8 neuronas y activación ReLU, y una capa de salida con 2 neuronas para predecir la dirección y el giro. El modelo se compila utilizando el optimizador Adam y la pérdida de error cuadrático medio (MSE).

```
[5]  # Definición del Modelo de Red Neuronal
     modelo = keras.Sequential([
         keras.layers.Dense(8, input_shape=(2,), activation='relu'),  # Capa oculta con 8 neuronas y activación ReLU
         keras.layers.Dense(8, activation='relu'),                    # Otra capa oculta con 8 neuronas y activación ReLU
         keras.layers.Dense(2)                                        # Capa de salida con 2 neuronas (dirección y giro)
     ])
```

Capas de la Red Neuronal

La función de Activación es ReLU se usa comúnmente en capas ocultas porque es más rápida de calcular y ayuda a mitigar el problema de desvanecimiento del gradiente.

El número de neuronas en las capas ocultas (8 en este caso) es una elección empírica y puede variar dependiendo de la complejidad del problema y la cantidad de datos disponibles. Se puede ajustar para obtener un equilibrio entre capacidad de representación y evitar el sobreajuste.

Capa de entrada

- ***"input_shape=(2,)"*** indica que se espera que cada muestra de entrada tenga dos características. Por lo tanto, la capa de entrada está compuesta por estas dos características: el sensor de distancia y la ubicación del obstáculo. Esencialmente, cada muestra de entrada se representa como un vector de dos elementos. Esto implica que la capa de entrada tiene una dimensión de 2.

Capa Oculta 1

- ***"keras.layers.Dense(8, input_shape=(2,), activation='relu')"*** esta es la primera capa oculta de la red neuronal.
- ***"keras.layers.Dense(8)"*** significa que esta capa tiene 8 neuronas.
- ***"input_shape=(2,)"*** indica que la capa espera una entrada de dos dimensiones, ya que hay dos características en los datos de entrada (sensor de distancia y ubicación del obstáculo).
- ***" activation='relu' "***: Utiliza la función de activación ReLU (Rectified Linear Unit), que es comúnmente utilizada en capas ocultas debido a su eficiencia y capacidad para tratar con el problema de desvanecimiento del gradiente.

Capa Oculta 2

- ***"keras.layers.Dense(8, activation='relu')"*** esta es la segunda capa oculta de la red neuronal. Tiene 8 neuronas y también utiliza la función de activación ReLU.

Capa de Salida

- ***"keras.layers.Dense(2)"*** esta es la capa de salida de la red neuronal. Tiene 2 neuronas, una para predecir la dirección y otra para predecir el giro. No se especifica ninguna función de activación aquí, lo que significa que esta capa realizará una regresión lineal para obtener las salidas.

- La función de Activación es ReLU se usa comúnmente en capas ocultas porque es más rápida de calcular y ayuda a mitigar el problema de desvanecimiento del gradiente.
- La capa de salida se eligen dos neuronas en la capa de salida porque el problema implica predecir dos valores: la dirección y el giro.

6. Compilar el modelo

```
[6]  # Compilar el modelo
     modelo.compile(optimizer='adam', loss='mean_squared_error')
```

- ***"modelo.compile( )"*** esta es una función de la API de Keras que se utiliza para compilar el modelo antes de entrenarlo. Compilar el modelo configura su proceso de aprendizaje, incluyendo el optimizador, la función de pérdida y las métricas que se utilizarán durante el entrenamiento y la evaluación.

- ***" optimizer='adam' "*** este parámetro especifica el algoritmo de optimización que se utilizará para ajustar los pesos del modelo durante el entrenamiento. Adam es un optimizador popular en el aprendizaje profundo que combina la idea de la tasa de aprendizaje adaptativa con el momento de Nesterov, lo que lo hace eficiente y fácil de usar.
- ***"loss='mean_squared_error' "*** este parámetro especifica la función de pérdida que se utilizará para evaluar el rendimiento del modelo durante el entrenamiento. En este caso, se utiliza el error cuadrático medio (MSE), que es una medida comúnmente utilizada para problemas de regresión. Calcula la media de las diferencias al cuadrado entre las predicciones del modelo y las etiquetas reales. Minimizar esta función de pérdida durante el entrenamiento significa que el modelo intentará hacer que las predicciones se acerquen lo más posible a las etiquetas reales.

7. Entrenamiento del modelo

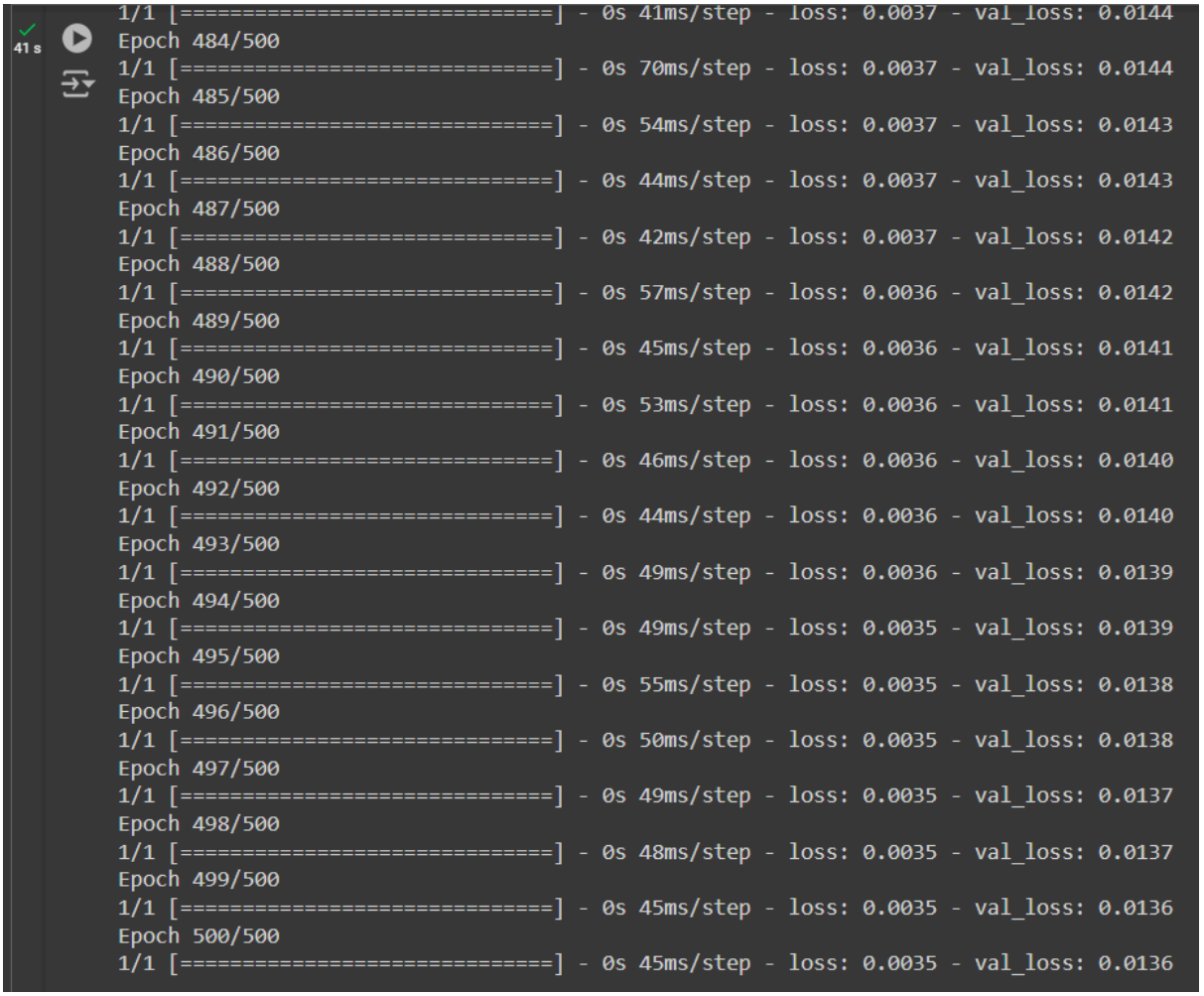
Se entrena el modelo utilizando los datos de entrenamiento. Se realizan 100 épocas de entrenamiento con un tamaño de lote de 10 y una división de validación del 20%. El historial del entrenamiento se almacena en la variable *"history"*.

```
#Entrenamiento del Modelo
history=modelo.fit(X_train, y_train, epochs=500, batch_size=10, validation_split=0.2)
```

- ***"modelo.fit( )"*** es un método que entrena el modelo de red neuronal. Toma los datos de entrenamiento *(`X_train` y `y_train`)* como entrada y ajusta los pesos del modelo para minimizar la función de pérdida.
- ***"X_train"*** y ***"y_train"*** son las características de entrenamiento y las etiquetas de entrenamiento respectivamente.
- ***"epochs=500"*** Número de épocas de entrenamiento. Una época se define como una pasada completa a través de todos los datos de entrenamiento. Aquí, se realizan 500 épocas de entrenamiento, lo que significa que el modelo verá todos los datos de entrenamiento 100 veces durante el entrenamiento.
- ***"batch_size=10"*** es el tamaño del lote utilizado para el entrenamiento. Durante el entrenamiento, los datos se dividen en lotes más pequeños, y los pesos del modelo se actualizan después de procesar cada lote. Un tamaño de lote de 10 significa que se utilizarán 10 muestras de datos para calcular el error y actualizar los pesos. Usar un tamaño de lote pequeño puede ayudar a la convergencia del modelo y permite que el entrenamiento sea más eficiente en memoria.
- ***"validation_split"***=0.2" es la fracción de los datos de entrenamiento que se utilizarán como conjunto de validación durante el entrenamiento. Aquí, el 20% de los datos de

entrenamiento se utilizarán para la validación. Esto significa que, durante cada época de entrenamiento, el modelo calculará la pérdida tanto en los datos de entrenamiento como en el conjunto de validación. La división en validación permite monitorear si el modelo está sobre ajustando los datos de entrenamiento y generalizando bien a datos no vistos. Esto es crucial para evitar el sobreajuste y para ajustar correctamente los hiper parámetros del modelo.

Entrenamiento:

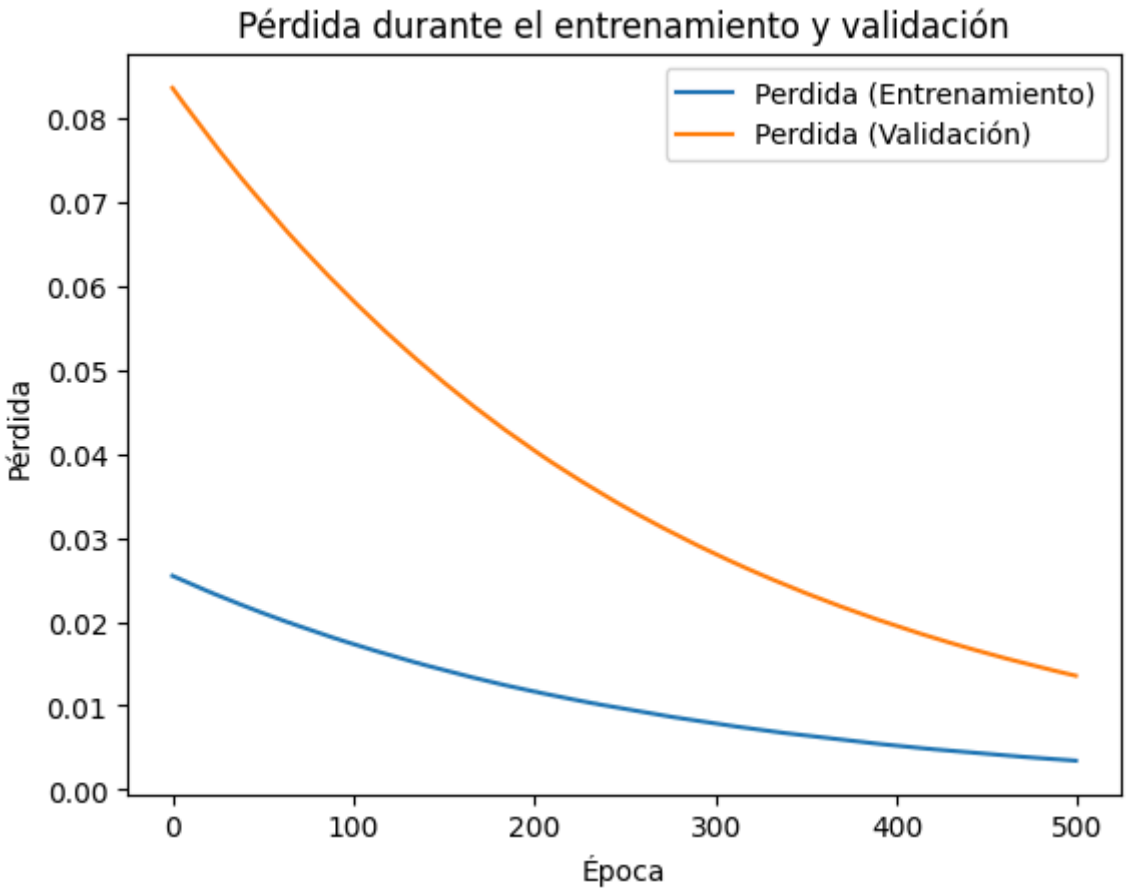


8. Gráfico de pérdida durante el entrenamiento

Se genera un gráfico que muestra la pérdida durante el entrenamiento y la validación a lo largo de las épocas.
Los valores de pérdida del conjunto de entrenamiento y de validación se utilizan para monitorear el desempeño del modelo durante el entrenamiento. La pérdida de entrenamiento indica cómo el modelo está aprendiendo de los datos de entrenamiento, mientras que la pérdida de validación indica cómo el modelo generaliza datos no vistos. Asignar estos valores en el gráfico nos permite visualizar cómo evoluciona la pérdida a lo largo del entrenamiento y la validación, lo que puede ayudar a identificar problemas como sobreajuste o subajuste.

```
[8]  # Gráfico de pérdida durante el entrenamiento
     plt.plot(history.history['loss'], label='Perdida (Entrenamiento)')
     plt.plot(history.history['val_loss'], label='Perdida (Validación)')
     plt.title('Pérdida durante el entrenamiento y validación')
     plt.xlabel('Época')
     plt.ylabel('Pérdida')
     plt.legend()
     plt.show()
```

- ***"plt.plot(history.history['loss'], label='Perdida (Entrenamiento)')"*** esta parte del código traza la pérdida en el conjunto de entrenamiento a lo largo de las épocas. ***"history.history['loss']"*** accede al diccionario ***"history"*** devuelto por el método ***"fit()"*** y extrae la lista de valores de pérdida del conjunto de entrenamiento. Estos valores se pasan como el primer argumento a ***"plt.plot()"***. Además, se proporciona una etiqueta para la leyenda del gráfico, que se mostrará como *"Pérdida (Entrenamiento)"*.
- ***"plt.plot(history.history['val_loss'], label='Perdida (Validación)')"*** esta parte traza la pérdida en el conjunto de validación a lo largo de las épocas. ***`history.history['val_loss']`*** accede al diccionario ***`history`*** y extrae la lista de valores de pérdida del conjunto de validación. Estos valores se pasan como el primer argumento a ***`plt.plot()`***. Se proporciona una etiqueta para la leyenda del gráfico, que se mostrará como *"Pérdida (Validación)"*.
- ***"plt.title('Pérdida durante el entrenamiento y validación')"*** Establece el título del gráfico como *"Pérdida durante el entrenamiento y validación"*.
- ***"plt.xlabel('Época')"*** : Etiqueta el eje x del gráfico como *"Época"*, lo que indica en qué época se encuentra cada punto de datos en el gráfico.
- ***"plt.ylabel('Pérdida')"***: Etiqueta el eje y del gráfico como *"Pérdida"*, indicando que los valores en el gráfico representan la pérdida del modelo.
- ***"plt.legend()"*** muestra la leyenda en el gráfico, que identifica qué línea representa la pérdida del entrenamiento y cuál representa la pérdida de validación.
- ***"plt.show()"*** Finalmente, muestra el gráfico.



Explicación del gráfico

Las curvas de pérdida de entrenamiento y validación no se cruzan y en caso de hacerlo sería en un número alto de valores lo cual indica que el modelo está aprendiendo de manera efectiva y generalizando bien.

9. Evaluación del modelo

Se evalúa el rendimiento del modelo utilizando los datos de prueba. La pérdida obtenida se imprime en la consola.

```
# Evaluación del Modelo
# Evaluamos el rendimiento del modelo en datos de prueba
loss = modelo.evaluate(X_test, y_test)
print(f'Loss: {loss}')
```

```
1/1 [==============================] - 0s 28ms/step - loss: 0.4445
Loss: 0.44452303647994995
```

- ***"modelo.evaluate(X_test, y_test)"*** Esta función `evaluate` toma el conjunto de características de prueba *"X_test"* y las etiquetas de prueba *"y_test"* como entrada y evalúa el rendimiento del modelo sobre estos datos de prueba.
- ***"loss"*** El resultado de *"modelo.evaluate"* es asignado a la variable *"loss"*, que representa la pérdida calculada por el modelo en el conjunto de prueba.

Asignación de los Valores

"X_test` y `y_test": Estos son los datos de prueba que se separaron anteriormente usando *"train_test_split"*. *"X_test"* contiene las características de prueba (lecturas del sensor), mientras que *"y_test"* contiene las etiquetas de prueba (acciones correspondientes).

Pérdida: La pérdida es una medida de cuánto difieren las predicciones del modelo de las etiquetas verdaderas. Es una métrica importante para evaluar el rendimiento del modelo. La pérdida suele ser calculada utilizando una función de pérdida específica, que en este caso es "*'mean_squared_error'"* (error cuadrático medio).

Razón de Asignar Tales Valores

Datos de Prueba: Es crucial evaluar el rendimiento del modelo en datos que no ha visto durante el entrenamiento para verificar su capacidad para generalizar a nuevos datos. *"X_test"* y *"y_test"* proporcionan este conjunto de datos de prueba separados.

Pérdida como Métrica de Rendimiento: La pérdida es una métrica comúnmente utilizada para evaluar el rendimiento del modelo. Cuanto menor sea la pérdida, mejor será el rendimiento del modelo en los datos de prueba. El error cuadrático medio es una opción común para problemas de regresión como este, donde estamos tratando de predecir valores continuos (dirección y giro).

10. Implementación en el carrito del juguete

La función `predecir` está diseñada para ser genérica y fácil de usar. Toma la lectura del sensor como entrada y devuelve la predicción del modelo correspondiente. La lectura del sensor se asigna directamente a la entrada del modelo después de convertirla en un arreglo NumPy para garantizar que tenga la forma correcta esperada por el modelo. Esto simplifica el proceso de hacer predicciones, ya que el usuario sólo necesita proporcionar la lectura del sensor y la función se encarga del resto, incluida la preparación de los datos y la realización de la predicción.

```
[10] # Implementación en el Carrito de Juguete
     # Función para predecir la acción (dirección y giro) basada en la lectura del sensor
     def predecir(sensor):
         return modelo.predict(np.array([sensor]))
```

Esta línea de código define una función llamada `predecir` que toma como entrada una lectura del sensor y devuelve la predicción del modelo para esa lectura específica. Aquí está la explicación detallada de cómo funciona y por qué se asignan los valores de esta manera:

- **Función "predecir":** Esta es una función simple que toma un único argumento, *`sensor`*, que representa la lectura actual del sensor. En esta función utiliza el modelo

previamente definido (`modelo`) para hacer una predicción sobre la acción correspondiente a esa lectura del sensor.

- **np.array([sensor]): "*np.array([sensor])"*** convierte la lectura del sensor en un arreglo NumPy de una dimensión. El uso de *`np.array`* asegura que la entrada sea un arreglo, que es el formato esperado por el modelo de TensorFlow para realizar predicciones. La lectura del sensor se envuelve en una lista *`[sensor]`* y luego se convierte en un arreglo NumPy para que tenga la forma adecuada para la entrada del modelo.
- **modelo.predict(...)** : ***"modelo.predict(...)"*** es un método que realiza una predicción utilizando el modelo de red neuronal previamente definido *(`modelo`)*. Además toma como argumento la entrada de datos en el formato adecuado para el modelo, que en este caso es la lectura del sensor convertida en un arreglo NumPy. Y finalmente devuelve la predicción del modelo, que en este caso sería la acción (dirección y giro) que el modelo recomienda tomar en función de la lectura del sensor proporcionada.

11. Ejemplo del uso

Este bloque de código se utiliza para probar el modelo de red neuronal entrenado con diferentes lecturas de sensores y observar las acciones predichas (dirección y giro).

El propósito de este bloque es permitir validar cómo el modelo responde a diferentes situaciones simuladas de la vida real, ayudar a entender qué acciones predice el modelo para diferentes entradas del sensor, evaluando si el modelo se comporta como se espera.y proveer ejemplos concretos que muestran el uso práctico del modelo en situaciones específicas, facilitando la evaluación y ajuste del modelo según sea necesario.

Este enfoque permite verificar si el modelo responde adecuadamente a las diferentes lecturas del sensor y realiza las acciones correctas para evitar obstáculos, lo que es crucial para aplicaciones en robótica y navegación autónoma.

```python
# Ejemplos de uso
print("Obstáculo a la izquierda")
lectura_sensor = [0.5, -1]  # Ejemplo de lectura del sensor (se acerca a un obstáculo a la izquierda)
accion = predecir(lectura_sensor)
print("Dirección:", accion[0][0])  # Imprimir dirección
print("Giro:", accion[0][1])       # Imprimir giro

print("Obstáculo en la parte delantera derecha")
lectura_sensor = [1, 1]  # Ejemplo de lectura del sensor con obstáculo en la parte delantera derecha
accion = predecir(lectura_sensor)
print("Dirección:", accion[0][0])  # Imprimir dirección
print("Giro:", accion[0][1])       # Imprimir giro

print("Sensor en posición central")
lectura_sensor = [0, 0]  # Ejemplo de lectura del sensor en posición central
accion = predecir(lectura_sensor)
print("Dirección:", accion[0][0])  # Imprimir dirección
print("Giro:", accion[0][1])       # Imprimir giro

print("Obstáculo demasiado cerca a la izquierda")
lectura_sensor = [1, -1]  # Ejemplo de lectura del sensor con obstáculo demasiado cerca a la izquierda
accion = predecir(lectura_sensor)
print("Dirección:", accion[0][0])  # Imprimir dirección
print("Giro:", accion[0][1])       # Imprimir giro

print("Obstáculo en la parte trasera izquierda")
lectura_sensor = [-1, -1]  # Ejemplo de lectura del sensor con obstáculo en la parte trasera izquierda
accion = predecir(lectura_sensor)
print("Dirección:", accion[0][0])  # Imprimir dirección
print("Giro:", accion[0][1])       # Imprimir giro
```

1. **Lectura del Sensor :** Cada lectura del sensor es una lista de dos elementos que representa la distancia y la ubicación del obstáculo.
2. **Función de Predicción :** La función `predecir(sensor)` toma la lectura del sensor como entrada, la convierte a un arreglo NumPy y utiliza el modelo entrenado para predecir las acciones (dirección y giro).
3. **Impresión de Resultados :** Después de realizar la predicción, se imprimen los valores de la dirección y el giro.

Implementación:

- **Entrada y Predicción**:
  - ***"lectura_sensor = [0.5, -1]"***: Se define una lectura de sensor, simulando una situación en la que el sensor detecta un obstáculo a una distancia de 0.5 unidades a la izquierda.
  - ***`accion = predecir(lectura_sensor)`***: Se llama a la función `predecir` con la lectura del sensor como argumento. Esta función utiliza el modelo entrenado para generar una predicción de las acciones (dirección y giro).
  - ***`print("Dirección:", accion[0][0])`***: Imprime la dirección predicha por el modelo.
  - ***`print("Giro:", accion[0][1])`***: Imprime el giro predicho por el modelo.

- **Repetición con Diferentes Entradas:** El proceso se repite para diferentes valores de `lectura_sensor` para simular diferentes escenarios:
  - Obstáculo a la izquierda.

- Obstáculo en la parte delantera derecha.
- Sensor en posición central.
- Obstáculo demasiado cerca a la izquierda.
- Obstáculo en la parte trasera izquierda.

Resultados:

```
Obstáculo a la izquierda
1/1 [==============================] - 0s 22ms/step
Dirección: -0.066352546
Giro: 0.68173933
Obstáculo en la parte delantera derecha
1/1 [==============================] - 0s 68ms/step
Dirección: -0.9910269
Giro: -1.0107703
Sensor en posición central
1/1 [==============================] - 0s 24ms/step
Dirección: 0.9037974
Giro: -0.46662426
Obstáculo demasiado cerca a la izquierda
1/1 [==============================] - 0s 20ms/step
Dirección: -1.0012124
Giro: 1.0011753
Obstáculo en la parte trasera izquierda
1/1 [==============================] - 0s 20ms/step
Dirección: 0.93094045
Giro: -0.35762227
```

12. Graficar las predicciones

Se genera un gráfico que muestra las predicciones del modelo junto con los datos de prueba. Esto proporciona una visualización de cómo el modelo interpreta la lectura del sensor y predice la acción correspondiente.

```python
# Graficar las predicciones
# Creamos un gráfico que muestre las predicciones junto con los datos de prueba
plt.scatter(X_test[:, 0], X_test[:, 1], label='Datos reales')
plt.scatter(lectura_sensor[0], lectura_sensor[1], color='red', label='Lectura del sensor')  # Marcar la lectura del sensor
plt.scatter(accion[0][0], accion[0][1], color='green', label='Predicción')  # Marcar la predicción
plt.xlabel('Sensor de distancia')
plt.ylabel('Ubicación del obstáculo')
plt.title('Predicción del modelo')
plt.legend()
plt.grid(True)
plt.show()
```

Esta línea de código genera un gráfico de dispersión que muestra los datos de prueba, la lectura actual del sensor y la predicción realizada por el modelo. Aquí está el desglose:

- **“plt.scatter(X_test[:, 0], X_test[:, 1], label='Datos reales')”**: Este comando crea un gráfico de dispersión utilizando los datos de prueba *`X_test`. `X_test[:, 0]`* y *`X_test[:, 1]`* son las coordenadas del sensor de distancia y la ubicación del obstáculo respectivamente. Estas son las características reales que el modelo usa para aprender. Se etiquetan como *"Datos reales"* en la leyenda del gráfico.

- **“plt.scatter(lectura_sensor[0], lectura_sensor[1], color='red', label='Lectura del sensor')”** : Agrega una marca roja para representar la lectura actual del sensor.

`lectura_sensor[0]` y *`lectura_sensor[1]`* son las coordenadas de la lectura del sensor actual. Esto proporciona una visualización de dónde está el obstáculo en relación con el sensor. Se etiqueta como *"Lectura del sensor"* en la leyenda.

- **“plt.scatter(accion[0][0], accion[0][1], color='green', label='Predicción')”** : Añade una marca verde para representar la predicción realizada por el modelo. *`accion[0][0]`* y *`accion[0][1]`* son las coordenadas de la predicción de dirección y giro respectivamente. Esto muestra qué acción tomará el modelo dada la lectura del sensor. Se etiqueta como *"Predicción"* en la leyenda.

- **“plt.xlabel('Sensor de distancia')”** : Etiqueta el eje x del gráfico como *"Sensor de distancia"*.

- **“plt.ylabel('Ubicación del obstáculo')”**: Etiqueta el eje y del gráfico como "Ubicación del obstáculo".

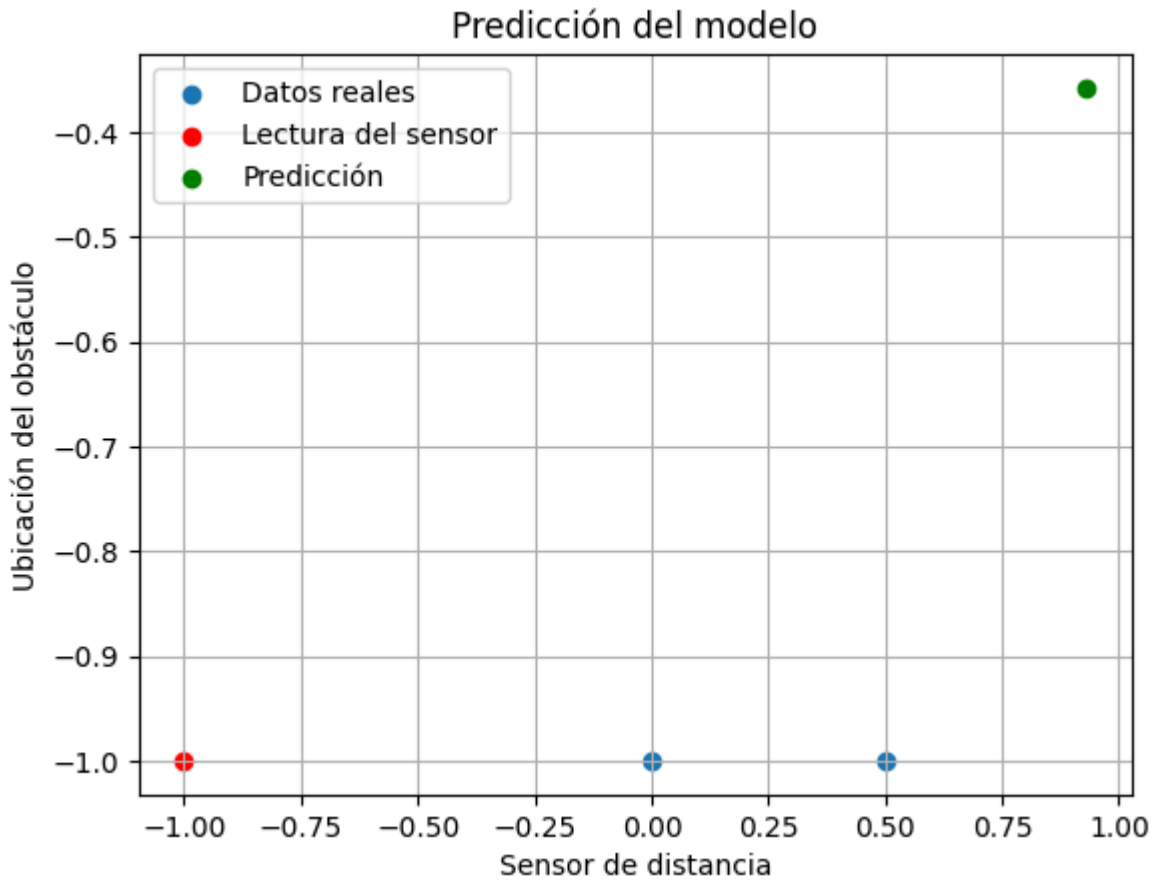
- **“plt.title('Predicción del modelo')”** establece el título del gráfico como "Predicción del modelo".

- **“plt.legend()”** : Muestra la leyenda en el gráfico que identifica qué puntos corresponden a los datos reales, la lectura del sensor y la predicción del modelo.

- **“plt.grid(True)”**: Añade una cuadrícula al gráfico para facilitar la referencia visual.

- **“plt.show()”**: Muestra el gráfico generado.

Se asignan estos valores para proporcionar una representación visual clara de los datos de prueba, la lectura del sensor actual y la predicción del modelo en relación con el sensor de distancia y la ubicación del obstáculo. La elección de colores diferentes y etiquetas claras ayuda a distinguir entre los diferentes elementos en el gráfico.



Explicación del gráfico:

- ***Lectura del sensor***: (-1.00, -1.00) indica que el carrito detecta un obstáculo muy cerca detrás y a la izquierda.
- ***Datos reales***: A y B son puntos de referencia usados durante el entrenamiento del modelo que indican situaciones en las cuales el obstáculo está cerca pero con una ligera variación en la posición.
- ***Predicción***: (0.90, -0.30) sugiere que el modelo predice una acción que probablemente implique moverse hacia adelante o hacia la derecha (valor positivo en la primera coordenada) mientras realiza un ligero giro hacia la izquierda (valor negativo en la segunda coordenada).

2. Red neuronal para predecir la diabetes en base a los atributos especificados.

Este código cubre todo el flujo desde la carga de datos hasta la evaluación y predicción usando un modelo de red neuronal en TensorFlow, además de la visualización de las métricas de rendimiento.

Explicación del código:

1. ***Importación de librerías***

```
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
```

- ***numpy:*** Biblioteca para trabajar con matrices y arrays.
- ***pandas:*** Biblioteca para manipulación y análisis de datos.
- ***tensorflow:*** Biblioteca de aprendizaje automático.
- ***Sequential y Dense:*** Componentes de Keras para construir modelos de redes neuronales.
- ***train_test_split:*** Función de Scikit-Learn para dividir datos en conjuntos de entrenamiento y prueba.
- ***matplotlib.pyplot:*** Biblioteca para crear gráficos.
- ***drive:*** Módulo para interactuar con Google Drive desde Google Colab.

2. ***Montar Google Drive en Colab***

```
from google.colab import drive
drive.mount("/content/drive",force_remount=True)
```

- Monta Google Drive en el entorno de Colab para acceder a los archivos almacenados allí.
- ***force_remount=True:*** Remonta Google Drive si ya está montado.

3. *Ruta al archivo de datos, carga y visualización de datos*

```
file_path = "/content/drive/MyDrive/Ramos - Torres: Evaluacion parcial Tecnologias emergentes/datos PACIENTE_DIABETES.xlsx"
raw_data=pd.read_excel(file_path,sheet_name=0) #indica que cargue la primera hoja del excel
dataset=raw_data.copy() #crea una copia del conjunto de datos original
dataset.head(7) #muestra las 7 primeras filas del Data frame
```

	Paciente	Fumador	Obesidad	Ejercicio	Genética	Diabético
0	Juan	0	0	0	0	0
1	Pedro	0	0	0	1	0
2	Marta	0	1	1	1	1
3	Rocío	0	0	1	1	0

- ***file path:*** Especifica la ruta del archivo Excel que contiene los datos.
- ***pd.read_excel(file_path, sheet_name=0):*** Carga los datos desde la primera hoja del archivo Excel.
- ***raw_data.copy():*** Crea una copia del DataFrame original.
- ***dataset.head(7):*** Muestra las primeras 7 filas del DataFrame.

4. *Leer datos desde un archivo Excel y preparar los datos*

```
df = pd.read_excel(file_path)

# Convertir los datos a formato numpy arrays
X = df[['Fumador', 'Obesidad', 'Ejercicio', 'Genética']].values
y = df['Diabético'].values
```

- ***df:*** Lee los datos del archivo Excel de nuevo (parece redundante, pero tal vez se hace por seguridad).
- ***X = df[['Fumador', 'Obesidad', 'Ejercicio', 'Genética']].values:*** Selecciona las columnas de características ('Fumador', 'Obesidad', 'Ejercicio', 'Genética') y las convierte a un array numpy (X).
- ***y = df['Diabético'].values:*** Selecciona la columna de la etiqueta ('Diabético') y la convierte a un array numpy (y).

5. *Dividir los datos en conjuntos de entrenamiento y prueba e Imprimir los conjuntos de datos*

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("Data de entrenamiento: \n",X_train)
print("Data de testeo: \n",X_test)
```

- ***train_test_split(X, y, test_size=0.2, random_state=42):*** Divide los datos en un 80% para entrenamiento y un 20% para prueba.
- ***random_state=42:*** Asegura que la división sea reproducible.
- ***print:*** Imprime los datos de entrenamiento y prueba para inspección.

6. *Definir el modelo y Compilar el modelo*

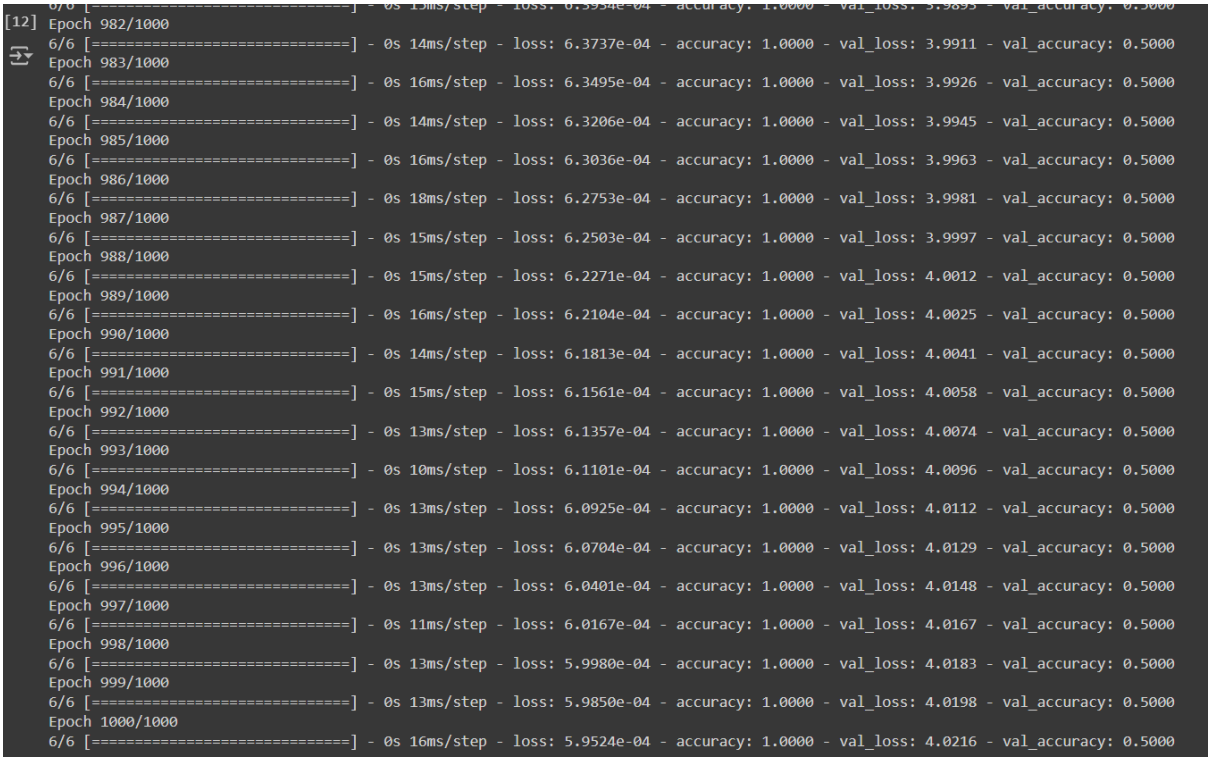
```
# Definir el modelo
model = Sequential([
    Dense(8, input_dim=4, activation='relu'),  # Capa oculta con 8 neuronas
    Dense(1, activation='sigmoid')             # Capa de salida con 1 neurona (predicción binaria)
])

# Compilar el modelo
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

- ***Sequential:*** Crea un modelo secuencial de Keras.
- ***Dense(8, input_dim=4, activation='relu'):*** Añade una capa densa con 8 neuronas, 4 entradas (una por característica), y función de activación ReLU.
- ***Dense(1, activation='sigmoid'):*** Añade una capa de salida con una neurona y función de activación sigmoide para predicción binaria.
- ***optimizer='adam':*** Usa el optimizador Adam.
- ***loss='binary_crossentropy':*** Usa la pérdida de entropía cruzada binaria.
- ***metrics=['accuracy']:*** Usa la precisión como métrica.

7. *Entrenar el modelo*

```
[12] # Entrenar el modelo y guardar el historial
     history = model.fit(X_train, y_train, epochs=1000, batch_size=1, validation_split=0.2)
```



- ***model.fit(...):*** Entrena el modelo con los datos de entrenamiento.
- ***epochs=100:*** Ejecuta 100 épocas de entrenamiento.
- ***batch_size=1:*** Usa un tamaño de lote de 1.
- ***validation_split=0.2:*** Usa el 20% de los datos de entrenamiento para validación.

8. Evaluar el modelo

```
# Evaluar el modelo en los datos de prueba
loss, accuracy = model.evaluate(X_test, y_test)
print(f'Loss: {loss}, Accuracy: {accuracy}')
```

```
1/1 [==========] - 0s 189ms/step - loss: 1.1146 - accuracy: 0.0000e+00
Loss: 1.1145753860473633, Accuracy: 0.0
```

- ***model.evaluate(X_test, y_test):*** Evalúa el modelo con los datos de prueba.
- ***print:*** Imprime la pérdida y precisión en los datos de prueba.

9. *Realizar predicciones*

```
# Realizar predicciones con el modelo
nuevos_datos = np.array([[1, 0, 0, 1], [0, 1, 1, 0]])
predicciones = model.predict(nuevos_datos)
```

```
1/1 [==============================] - 0s 91ms/step
```

- ***np.array(...):*** Crea un array numpy con nuevos datos para predicción.
- ***model.predict(nuevos_datos):*** Genera predicciones para los nuevos datos.

10. *Convertir predicciones a binario*

```
# Convertir predicciones a binario (0 o 1) y añadir texto explicativo
predicciones_binarias = (predicciones > 0.5).astype(int)
for i, pred in enumerate(predicciones_binarias):
    estado = "diabético" if pred[0] == 1 else "no diabético"
    print(f'Paciente {i+1}: {estado}')
```

```
Paciente 1: diabético
Paciente 2: diabético
```

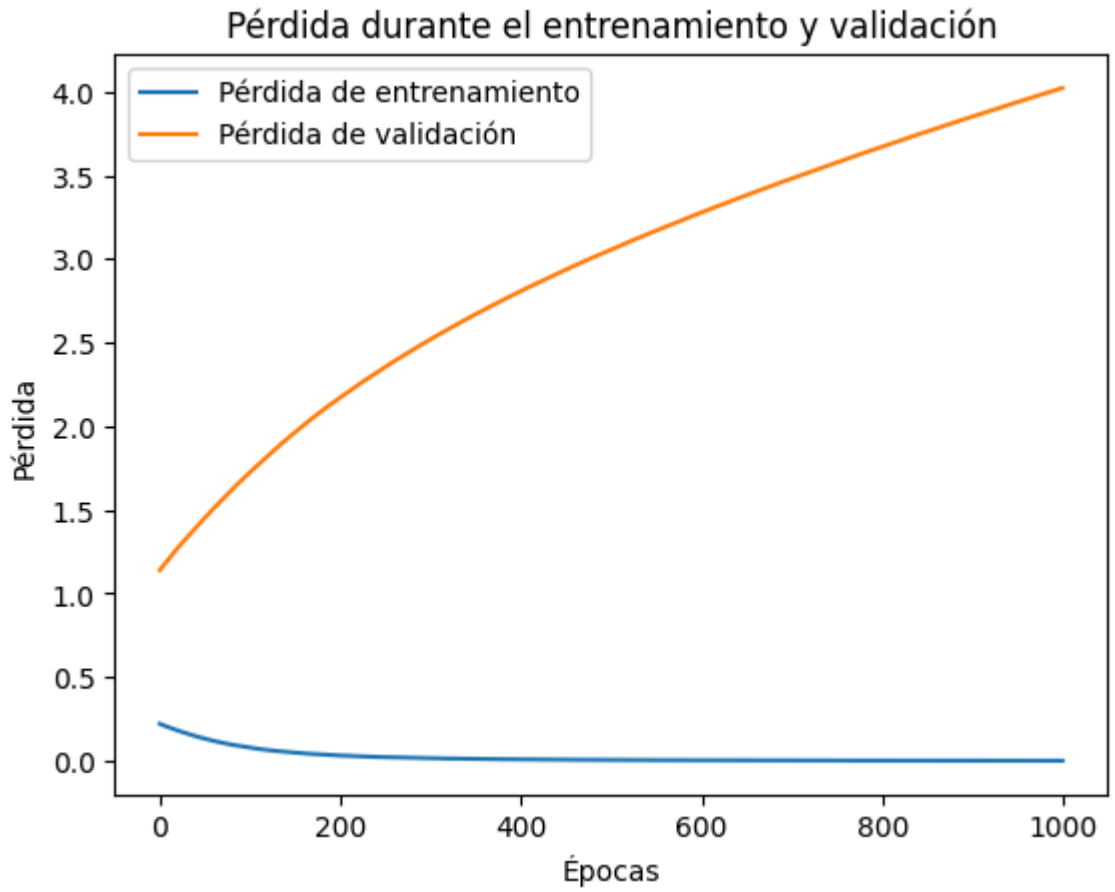
- ***(predicciones > 0.5).astype(int):*** Convierte las predicciones a binario (0 o 1).
- Itera sobre las predicciones y muestra si cada paciente es diabético o no.

11. *Graficar las pérdidas:*

```
# Graficar las pérdidas (loss) durante el entrenamiento
plt.plot(history.history['loss'], label='Pérdida de entrenamiento')
plt.plot(history.history['val_loss'], label='Pérdida de validación')
plt.xlabel('Épocas')
plt.ylabel('Pérdida')
plt.legend()
plt.title('Pérdida durante el entrenamiento y validación')
plt.show()
```

- ***plt.plot(...):*** Llama a la función plot de matplotlib.pyplot para crear un gráfico de líneas.
- ***history.history['loss']:*** Accede a los valores de pérdida (loss) del entrenamiento almacenados en el diccionario history.history durante cada época de entrenamiento. history es un objeto que almacena el historial del entrenamiento y es devuelto por el método fit de Keras.

- ***label='Pérdida de entrenamiento':*** Añade una etiqueta a la línea del gráfico para identificarla como la pérdida del entrenamiento.
- ***history.history['val_loss']:*** Accede a los valores de pérdida (loss) de la validación almacenados en el diccionario history.history durante cada época de validación.
- ***label='Pérdida de validación':*** Añade una etiqueta a la línea del gráfico para identificarla como la pérdida de validación.
- ***plt.xlabel('Épocas'):*** Etiqueta el eje X del gráfico con el texto "Épocas", indicando que el eje X representa el número de épocas de entrenamiento.
- ***plt.ylabel('Pérdida'):*** Etiqueta el eje Y del gráfico con el texto "Pérdida", indicando que el eje Y representa el valor de la pérdida.
- ***plt.legend():*** Añade una leyenda al gráfico que mostrará las etiquetas de las líneas definidas anteriormente ('Pérdida de entrenamiento' y 'Pérdida de validación').
- ***plt.title('Pérdida durante el entrenamiento y validación'):*** Establece el título del gráfico, describiendo claramente lo que se muestra en el gráfico.
- ***plt.show():*** Muestra el gráfico en la pantalla. Esta función se utiliza para renderizar el gráfico y hacerlo visible.



Explicación del gráfico
Las curvas de pérdida de entrenamiento y validación nunca se cruzan, lo cual indica que el modelo está aprendiendo de manera efectiva y generalizando bien.

3. Clasificación binaria

Este modelo está diseñado para construir y evaluar un modelo de clasificación binaria utilizando el conjunto de datos Breast Cancer Wisconsin. El propósito es entrenar un modelo de regresión logística para predecir si un tumor es maligno o benigno basado en varias características medidas en imágenes digitales de células aspiradas por aguja fina de una mama.

Explicacion del codigo:

1. ***Importar las bibliotecas***

```
[1]  # Importar las bibliotecas necesarias
     from sklearn.datasets import load_breast_cancer
     from sklearn.model_selection import train_test_split
     from sklearn.linear_model import LogisticRegression
     from sklearn.metrics import classification_report
     import pandas as pd
     import matplotlib.pyplot as plt
     import seaborn as sns
     from sklearn.metrics import confusion_matrix
```

- ***from sklearn.datasets import load_breast_cancer:*** Importa la función load_breast_cancer de scikit-learn para cargar el conjunto de datos de cáncer de mama.
- ***from sklearn.model_selection import train_test_split:*** Importa la función train_test_split para dividir el conjunto de datos en entrenamiento y prueba.
- ***from sklearn.linear_model import LogisticRegression:*** Importa el modelo de regresión logística.
- ***from sklearn.metrics import classification_report:*** Importa la función classification_report para generar un informe de clasificación.
- ***import pandas as pd:*** Importa la biblioteca pandas, utilizada para manipulación y análisis de datos.

- ***import matplotlib.pyplot as plt:*** Importa la biblioteca Matplotlib para crear gráficos.
- ***import seaborn as sns:*** Importa Seaborn para visualizaciones estadísticas más atractivas.
- ***from sklearn.metrics import confusion_matrix:*** Importa la función confusion_matrix para calcular la matriz de confusión.

2. Carga del dataset

El código carga el conjunto de datos Breast Cancer Wisconsin y lo convierte en estructuras de datos de pandas para facilitar su manipulación y análisis. Las características se almacenan en un DataFrame con nombres de columna descriptivos, y las etiquetas se almacenan en una serie. Esto prepara los datos de manera conveniente para el análisis y la modelización posterior.

```
[2]  # Carga del dataset
     cancer = load_breast_cancer()
     X = pd.DataFrame(cancer.data, columns=cancer.feature_names)
     y = pd.Series(cancer.target)
```

- ***cancer = load_breast_cancer():*** Carga el conjunto de datos de cáncer de mama en la variable cancer.
- ***X = pd.DataFrame(cancer.data, columns=cancer.feature_names):*** Convierte los datos de las características (cancer.data) en un DataFrame de pandas X con los nombres de las características (cancer.feature_names) como columnas.
- ***y = pd.Series(cancer.target):*** Convierte las etiquetas de los datos (cancer.target) en una Serie de pandas y.
    - ***cancer.target:*** Este es un array de NumPy que contiene las etiquetas (objetivo) del conjunto de datos. Cada elemento del array es una etiqueta binaria que indica si la muestra es benigna (0) o maligna (1).
    - ***pd.Series: "pandas.Series"*** es una estructura de datos unidimensional de la biblioteca pandas que permite manipular datos con una sola etiqueta (nombre).
    - ***Resultado: "y"*** es una serie de ***"pandas"*** que contiene las etiquetas (objetivo) para cada muestra. Esto facilita la manipulación y el análisis de las etiquetas junto con las características en X

3. *División de los datos*

```
[3] # Dividir el conjunto de datos en entrenamiento (80%) y prueba (20%)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- Divide el conjunto de datos en datos de entrenamiento (80%) y datos de prueba (20%) utilizando `**train_test_split**`.. El parámetro random_state=42 asegura que la división sea reproducible.

4. *Entrenamiento del Modelo*

Este proceso es esencial para entrenar modelos de clasificación y asegurar que se ajusten adecuadamente a los datos, logrando así hacer predicciones precisas en nuevos conjuntos de datos.

```
[4] # Inicializar y entrenar el modelo de clasificación utilizando Logistic Regression
    model = LogisticRegression(max_iter=10000)
    model.fit(X_train, y_train)
```

```
LogisticRegression(max_iter=10000)
```

- ***model = LogisticRegression(max_iter=10000):*** Inicializa el modelo de regresión logística con un máximo de 10000 iteraciones para asegurar la convergencia.
    - **LogisticRegression**: Esta es una clase en ***"scikit-learn"*** que implementa la regresión logística. La regresión logística es un modelo estadístico que se utiliza para realizar tareas de clasificación binaria (aunque también puede extenderse a la clasificación multiclase).
    - **max_iter=10000**: Este parámetro establece el número máximo de iteraciones para el algoritmo de optimización. La regresión logística en ***"scikit-learn"*** se ajusta utilizando un algoritmo de optimización iterativo (como el descenso de gradiente). A veces, los datos pueden ser complejos y el algoritmo necesita muchas iteraciones para converger a una solución. Al establecer ***"max_iter"*** en 10000, aseguramos que el algoritmo tiene suficiente oportunidad para converger.
- ***model.fit(X_train, y_train):*** Entrena el modelo de regresión logística usando los datos de entrenamiento X_train y y_train.
    - X_train: Las características (features) del conjunto de datos de entrenamiento.
    - y_train: Las etiquetas (targets) correspondientes a X_train.
- El método fit ajusta el modelo a los datos de entrenamiento, encontrando los parámetros que mejor se ajustan a los datos. Durante este proceso, el algoritmo de

optimización ajusta los coeficientes del modelo para minimizar el error de predicción en los datos de entrenamiento.

¿Por Qué Usar max_iter=10000?

1. Convergencia:
    - El algoritmo de optimización utilizado por LogisticRegression (por ejemplo, el descenso de gradiente) puede requerir múltiples iteraciones para converger a una solución óptima, especialmente si los datos son complejos o si hay muchas características. A veces, el número predeterminado de iteraciones (por ejemplo, 100 en algunas implementaciones) no es suficiente para que el algoritmo alcance la convergencia, lo que puede llevar a un modelo mal ajustado.
    - Establecer max_iter=10000 garantiza que el algoritmo tenga suficiente oportunidad para encontrar la mejor solución, reduciendo la probabilidad de que se detenga prematuramente antes de alcanzar la convergencia.
2. Complejidad de los Datos:
    - Si los datos tienen muchas características o si las relaciones entre las características y las etiquetas son complejas, el algoritmo puede necesitar más iteraciones para ajustar los parámetros del modelo adecuadamente.
    - En tales casos, un mayor número de iteraciones puede ayudar a encontrar una solución más precisa y estable.

5. Predicciones y Evaluación

```
[5]  # Predecir las etiquetas en el conjunto de prueba
     y_pred = model.predict(X_test)

[6]  # Evaluar el modelo utilizando clasificación_report que incluye precisión, recall y F1-score
     report = classification_report(y_test, y_pred)
```

- ***y_pred = model.predict(X_test):*** Predice las etiquetas para los datos de prueba X_test utilizando el modelo entrenado.

- ***report = classification_report(y_test, y_pred):*** Genera un informe de clasificación que incluye precisión, recall, F1-score y soporte para cada clase.

    - La función ***"classification_report"*** compara las etiquetas verdaderas (*"y_test"*) con las predicciones del modelo (*"y_pred"*) y genera un informe que incluye varias métricas de evaluación, como precisión, recall, F1-score y soporte para cada clase.
    - Los valores de ***"y_test"*** y ***"y_pred"***se utilizan para calcular estas métricas y generar el informe.
    - El informe generado se asigna a la variable report para su posterior uso, como la impresión o la visualización.

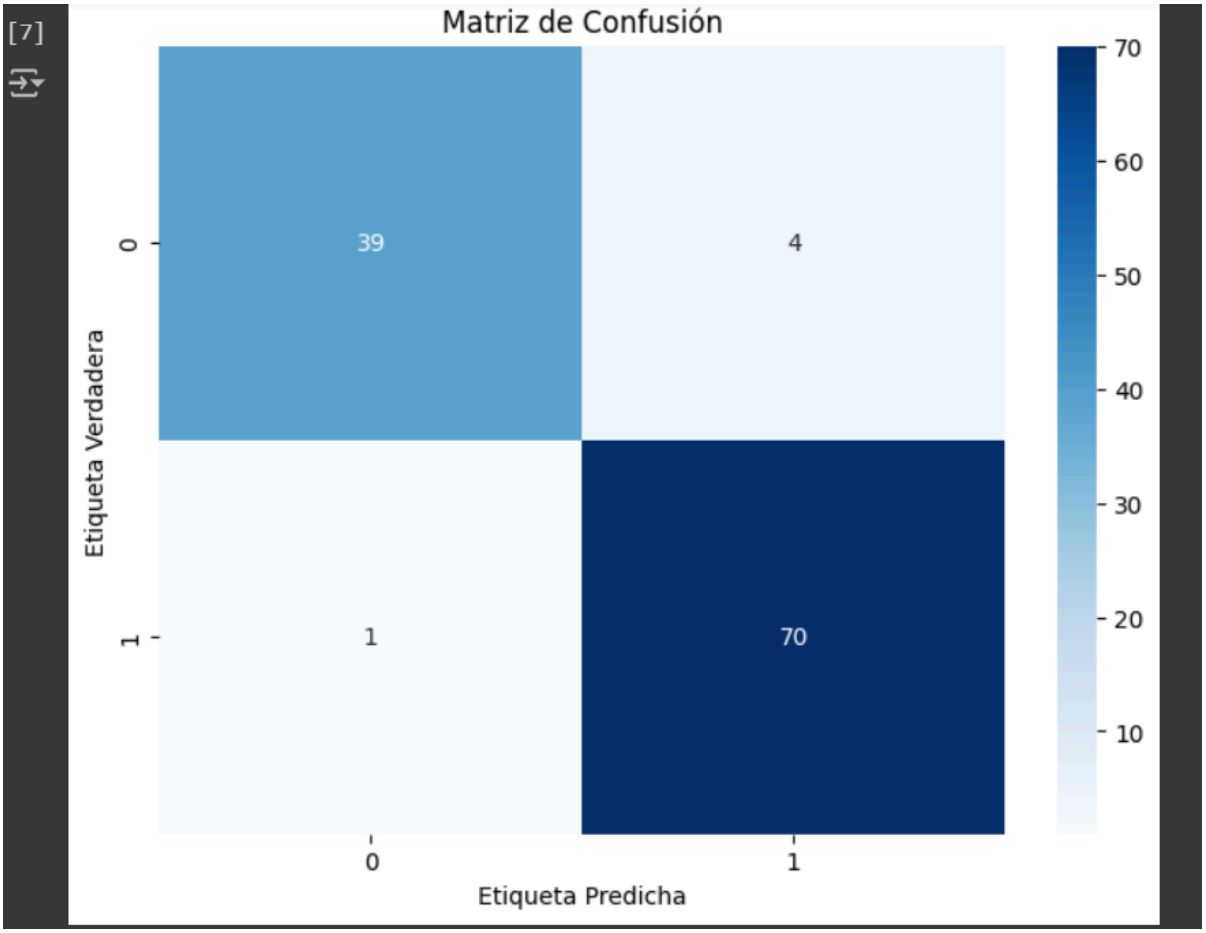
El motivo de asignar estos valores es evaluar el rendimiento del modelo en el conjunto de prueba. La precisión, recall y F1-score son métricas comunes utilizadas en problemas de clasificación para medir qué tan bien el modelo puede predecir correctamente las clases de interés. Al asignar las predicciones del modelo a *"y_pred"* y las etiquetas verdaderas a *"y_test"*, podemos utilizar estas variables para calcular estas métricas y obtener una evaluación cuantitativa del rendimiento del modelo.

6. *Matriz de confusión*

Este bloque de código crea una representación visual de la matriz de confusión utilizando un mapa de calor, lo que facilita la interpretación de las predicciones del modelo en comparación con las etiquetas verdaderas. Los valores en cada celda de la matriz indican el número de instancias que se clasificaron correctamente o incorrectamente. Los parámetros utilizados se eligen para mejorar la legibilidad y comprensión del gráfico.

```
[7]  # Matriz de confusión
     conf_matrix = confusion_matrix(y_test, y_pred)

     plt.figure(figsize=(8, 6))
     sns.heatmap(conf_matrix, annot=True, cmap='Blues', fmt='g')
     plt.title('Matriz de Confusión')
     plt.xlabel('Etiqueta Predicha')
     plt.ylabel('Etiqueta Verdadera')
     plt.show()
```

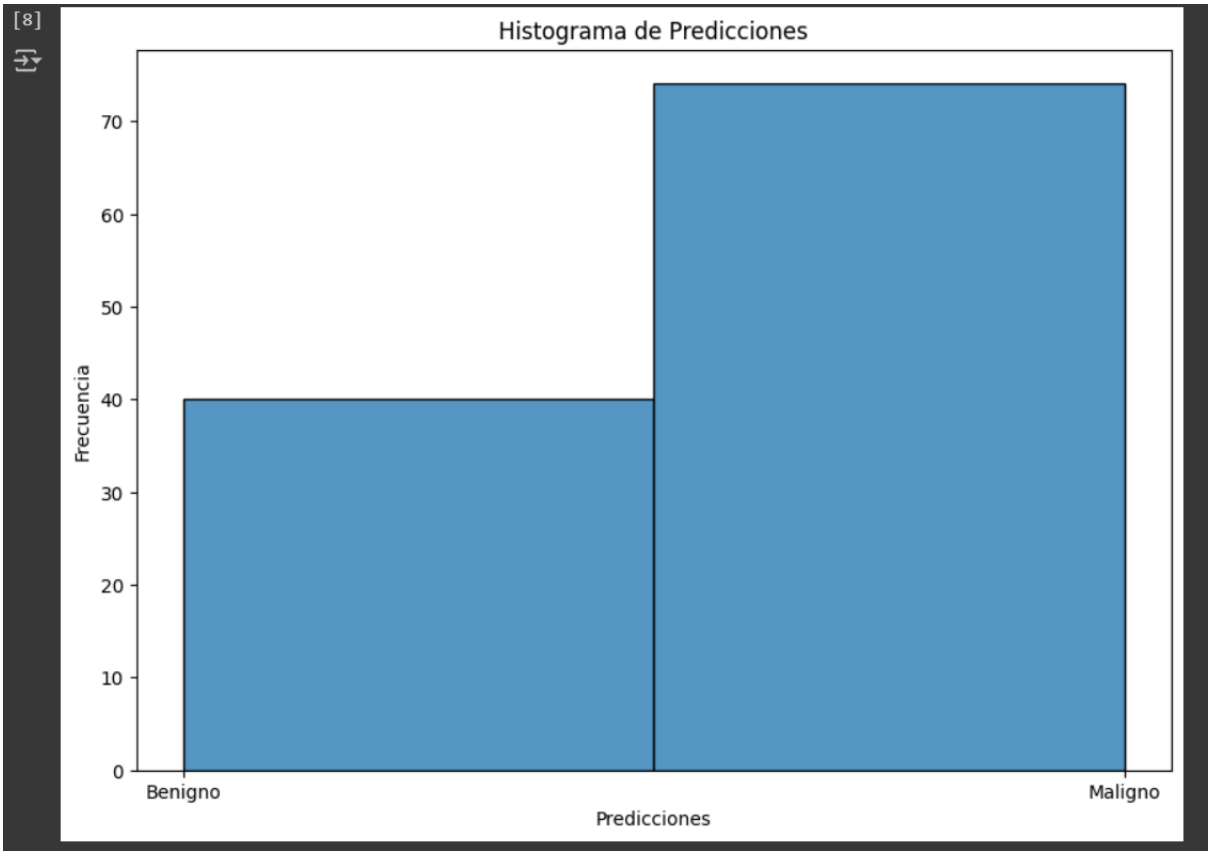


- ***conf_matrix = confusion_matrix(y_test, y_pred):*** Calcula la matriz de confusión para los datos de prueba y_test y las predicciones y_pred.
    - ***confusion_matrix(y_test, y_pred)***: La función confusion_matrix toma como entrada las etiquetas verdaderas *("y_test")* y las predicciones del modelo *("y_pred")* y calcula la matriz de confusión. La matriz de confusión es una tabla que muestra el número de predicciones correctas e incorrectas hechas por el clasificador.
- ***plt.figure(figsize=(8, 6)):*** Crea una nueva figura con un tamaño específico.El tamaño de la figura se especifica en pulgadas, donde el primer número representa el ancho y el segundo número representa la altura.
- ***sns.heatmap(conf_matrix, annot=True, cmap='Blues', fmt='g'):*** Usa Seaborn para crear un mapa de calor de la matriz de confusión con anotaciones (valores en las celdas), usando el esquema de colores 'Blues'.
- ***plt.title('Matriz de Confusión'):*** Añade un título al gráfico.
- ***plt.xlabel('Etiqueta Predicha'):*** Añade una etiqueta al eje X.
- ***plt.ylabel('Etiqueta Verdadera'):*** Añade una etiqueta al eje Y.
- ***plt.show():*** Muestra el gráfico.

7. *Histograma de predicciones*

El histograma de predicciones es una forma útil de visualizar la distribución de las predicciones realizadas por un modelo de clasificación binaria. En este caso, se utiliza para mostrar la frecuencia de las predicciones de las clases "benigno" y "maligno" en el conjunto de pruebas.

```
[8]  # Histograma de predicciones
     predictions = pd.Series(y_pred)
     plt.figure(figsize=(10,7))
     sns.histplot(predictions, bins=2, kde=False)
     plt.xlabel('Predicciones')
     plt.ylabel('Frecuencia')
     plt.title('Histograma de Predicciones')
     plt.xticks([0, 1], ['Benigno', 'Maligno'])
     plt.show()
```



- ***predictions = pd.Series(y_pred):*** Convierte las predicciones y_pred en una Serie de pandas.
- ***plt.figure(figsize=(10,7)):*** Crea una nueva figura con un tamaño específico.

- ***sns.histplot(predictions, bins=2, kde=False):*** Usa Seaborn para crear un histograma de las predicciones con 2 bins (para las dos clases: benigno y maligno), sin la estimación de densidad de Kernel (kde=False).
- ***plt.xlabel('Predicciones'):*** Añade una etiqueta al eje X.
- ***plt.ylabel('Frecuencia'):*** Añade una etiqueta al eje Y.
- ***plt.title('Histograma de Predicciones'):*** Añade un título al gráfico.
- ***plt.xticks([0, 1], ['Benigno', 'Maligno']):*** Configura las etiquetas de los ticks en el eje X para mostrar 'Benigno' y 'Maligno' en lugar de 0 y 1.
- ***plt.show():*** Muestra el gráfico.

8. ***Imprimir resultados***

El `classification_report` proporciona métricas de evaluación detalladas del rendimiento del clasificador en forma de texto. Estas métricas incluyen precisión, recall, F1-score y soporte para cada clase, así como la precisión global, recall y F1-score del modelo.

La razón de imprimir este informe es proporcionar una visión general del rendimiento del modelo de clasificación. Esto permite una fácil interpretación de cómo el modelo se desempeña en términos de precisión (la proporción de predicciones correctas), recall (la proporción de casos positivos reales que se identifican correctamente) y F1-score (la media ponderada de precisión y recall). Además, el informe también muestra el soporte, que es el número de ocurrencias de cada clase en el conjunto de prueba.

El reporte impreso es útil para comprender cómo el modelo se comporta en la clasificación de las clases benignas y malignas en el conjunto de datos Breast Cancer Wisconsin.

```
[9]  # Imprimir los resultados
     print(report)
```

```
              precision    recall  f1-score   support

           0       0.97      0.91      0.94        43
           1       0.95      0.99      0.97        71

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```

- ***print(report):*** Imprime el informe de clasificación que incluye precisión, recall, F1-score y soporte para cada clase.

4. Modelo de clasificación de imágenes utilizando una red neuronal convolucional (CNN).

Este código cubre todo el flujo desde la carga y preprocesamiento de datos, construcción, entrenamiento y evaluación de un modelo CNN, hasta la predicción de nuevas imágenes usando el modelo entrenado.

Explicacion del codigo:

1. ***Importación de librerías***

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.preprocessing.image import load_img, img_to_array
```

- ***import numpy as np:*** Importa la biblioteca NumPy, que se utiliza para manejar arrays y realizar operaciones matemáticas.
- ***import matplotlib.pyplot as plt:*** Importa la biblioteca Matplotlib para crear gráficos.
- ***from tensorflow.keras.datasets import cifar10:*** Importa el módulo cifar10 de Keras, que contiene el conjunto de datos CIFAR-10.
- ***from tensorflow.keras.models import Sequential:*** Importa el modelo secuencial de Keras, que permite construir una red neuronal capa por capa.
- ***from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense:*** Importa las capas necesarias para construir la red neuronal:
- ***Conv2D:*** Capa convolucional.
- ***MaxPooling2D:*** Capa de max-pooling.
- ***Flatten:*** Capa que aplana la entrada.
- ***Dense:*** Capa totalmente conectada.
- ***from tensorflow.keras.utils import to_categorical:*** Importa la función to_categorical para convertir etiquetas a formato one-hot.

- ***from tensorflow.keras.preprocessing.image import load_img, img_to_array:*** Importa funciones para cargar y convertir imágenes.

2. Carga del dataset CIFAR-10

Esta línea del código carga y divide el conjunto de datos CIFAR-10 en conjuntos de entrenamiento y prueba, lo que permite entrenar y evaluar modelos de aprendizaje automático de manera efectiva. Esto es crucial para desarrollar y validar modelos de visión por computadora y asegurar su capacidad de generalización.

```
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
```

a) cifar10.load_data()

- La función `load_data()` es un método proporcionado por TensorFlow Keras que carga el conjunto de datos CIFAR-10 desde su origen, que puede ser un archivo local o una ubicación en línea.
- El conjunto de datos CIFAR-10 es un conjunto de imágenes etiquetadas que se utiliza comúnmente para entrenar y evaluar modelos de aprendizaje automático, especialmente en el campo de la visión por computadora.

b) (x_train, y_train), (x_test, y_test) = cifar10.load_data()

- Esta línea de código asigna los datos cargados a cuatro variables: `x_train`, `y_train`, `x_test`, y `y_test`.
- `x_train` contiene las imágenes de entrenamiento, mientras que `y_train` contiene las etiquetas correspondientes para esas imágenes.
- Del mismo modo, `x_test` contiene las imágenes de prueba y `y_test` contiene las etiquetas correspondientes para las imágenes de prueba.

Razón para realizar esto:

a) División de datos de entrenamiento y prueba:

- La división de los datos en conjuntos de entrenamiento y prueba es una práctica estándar en el aprendizaje automático.
- El conjunto de entrenamiento se utiliza para entrenar el modelo, mientras que el conjunto de prueba se utiliza para evaluar su rendimiento en datos no vistos.

b) Evaluación del modelo:

- Al separar los datos en conjuntos de entrenamiento y prueba, podemos evaluar el rendimiento del modelo en datos que no ha visto durante el entrenamiento.
- Esto nos proporciona una estimación más precisa de la capacidad de generalización del modelo.

c) Prevención del sobreajuste:

- Al evaluar el modelo en datos de prueba independientes, podemos detectar si el modelo se ha sobreajustado a los datos de entrenamiento.
- El sobreajuste ocurre cuando el modelo se adapta demasiado a los datos de entrenamiento y no generaliza bien a nuevos datos.

3. Normalización de los datos

Este bloque de código convierte los valores de píxeles de las imágenes a flotantes de 32 bits y los normaliza al rango [0, 1], lo que prepara los datos para ser utilizados en el entrenamiento de un modelo de red neuronal convolucional para la clasificación de imágenes.

```
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
```

Acciones importantes:

a) ***Conversión de tipo de datos a 'float32':*** `x_train` y `x_test` son los conjuntos de datos de entrenamiento y prueba, respectivamente. Estos conjuntos contienen valores de píxeles de imágenes representados como enteros de 8 bits (valores en el rango de 0 a 255). Sin embargo, para realizar operaciones matemáticas eficientes y mantener la precisión de los cálculos en el entrenamiento de modelos de redes neuronales, es común convertir estos valores a flotantes de 32 bits.

b) ***Normalización de los datos:*** Después de convertir los valores de píxeles a flotantes, se realiza la normalización dividiendo cada valor de píxel por 255.0. La normalización es una práctica común en el preprocesamiento de datos en el aprendizaje profundo. Normalizar los valores de los píxeles asegura que todos los valores estén en un rango comparable (entre 0 y 1 en este caso), lo que ayuda a mejorar la estabilidad y la velocidad de convergencia del modelo durante el entrenamiento. Además, puede ayudar a evitar problemas como el estancamiento en ciertas partes de la función de pérdida.

4. Conversión de etiquetas a one-hot encoding

Convierte las etiquetas de las imágenes a formato one-hot encoding. Esto es necesario para la clasificación multiclase. El segundo argumento 10 indica que hay 10 clases.

La codificación one-hot es una técnica utilizada para representar etiquetas categóricas en forma de vectores binarios. Cada etiqueta se convierte en un vector donde un solo elemento es 1 (activo) y todos los demás son 0 (inactivos). Esto es útil en problemas de clasificación donde las etiquetas no tienen una relación de orden inherente y son discretas

```
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

Funcionamiento de `to_categorical`:

a) Entradas

- ***`y_train`:*** Un array que contiene las etiquetas de clase del conjunto de datos de entrenamiento.
- ***`y_test`:*** Un array que contiene las etiquetas de clase del conjunto de datos de prueba.
- ***`10`***: El número total de clases en el problema de clasificación (en este caso, CIFAR-10 tiene 10 clases).

b) Proceso

- La función ***`to_categorical`*** toma las etiquetas de clase como entrada y devuelve una representación one-hot de esas etiquetas.
- Cada etiqueta se convierte en un vector binario con la longitud igual al número total de clases.
- El valor correspondiente a la clase de la etiqueta es 1 y todos los demás valores son 0.

c) Razón para realizar esto

- En el *aprendizaje supervisado*, los modelos de machine learning necesitan etiquetas numéricas para realizar cálculos y entrenar de manera efectiva.
- Las etiquetas originales son simplemente números enteros que representan las clases (por ejemplo, 0 para 'avión', 1 para 'automóvil', etc. en CIFAR-10).
- Convertir las etiquetas en una representación one-hot proporciona una forma de representar las clases de manera más adecuada para el modelo.
- Esto ayuda al modelo a comprender que las clases son mutuamente excluyentes y no tienen una relación de orden.
- También facilita el cálculo de la pérdida y las métricas durante el entrenamiento y la evaluación del modelo.

5. Construcción del modelo CNN

Este modelo sigue una arquitectura común para problemas de clasificación de imágenes con CNNs. Las capas convolucionales se utilizan para extraer características espaciales de las imágenes, seguidas de capas de agrupamiento para reducir la dimensionalidad y capas densas para realizar la clasificación final basada en estas características. La función de activación relu se utiliza ampliamente debido a su eficacia en la introducción de no linealidad en la red.

La capa de salida utiliza softmax para producir una distribución de probabilidad sobre las clases.

```
model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(Flatten())
model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))
```

- ***model = Sequential():*** Crea un modelo secuencial, que es una pila lineal de capas. Este tipo de modelo es adecuado para la mayoría de las aplicaciones de redes neuronales donde las capas se añaden secuencialmente una tras otra.

- ***model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))***: Añade una capa convolucional bidimensional al modelo.

  32: El número de filtros (o canales de salida) en la convolución. Cada filtro aprende una característica distinta de la imagen.

  (3, 3): El tamaño de cada filtro es de 3x3 píxeles.

  activation='relu': La función de activación ReLU (Rectified Linear Unit) se aplica a la salida de cada filtro.

  input_shape=(32, 32, 3): Define la forma de entrada de la imagen. En este caso, las imágenes tienen un tamaño de 32x32 píxeles y 3 canales de color (RGB).

- ***model.add(MaxPooling2D((2, 2))):*** Añade una capa de max-pooling bidimensional al modelo.

  (2, 2): El tamaño del filtro de pooling es de 2x2 píxeles, lo que reduce cada dimensión espacial de la entrada a la mitad.

- ***model.add(Conv2D(64, (3, 3), activation='relu')):*** Añade una segunda capa convolucional bidimensional.

  64: El número de filtros en esta capa es de 64.

  (3, 3): El tamaño de cada filtro es de 3x3 píxeles.

  activation='relu': La función de activación ReLU se aplica a la salida de cada filtro.

- ***model.add(MaxPooling2D((2, 2))):*** Añade otra capa de max-pooling bidimensional al modelo.

(2, 2): El tamaño del filtro de pooling es de 2x2 píxeles.

- ***model.add(Conv2D(64, (3, 3), activation='relu')):*** Añade una tercera capa convolucional bidimensional.

  64: El número de filtros en esta capa es de 64.

  (3, 3): El tamaño de cada filtro es de 3x3 píxeles.

  activation='relu': La función de activación ReLU se aplica a la salida de cada filtro.

- ***model.add(Flatten()):*** Añade una capa de aplanamiento. Esta capa convierte la salida tridimensional de las capas anteriores en un vector unidimensional. Esto es necesario para conectar las capas convolucionales con las capas densamente conectadas (fully connected) que siguen.

- ***model.add(Dense(64, activation='relu')):*** Añade una capa densa con 64 neuronas.

  64: El número de neuronas en esta capa.

  activation='relu': La función de activación ReLU se aplica a la salida de cada neurona.

- ***model.add(Dense(10, activation='softmax')):*** Añade la capa de salida con 10 neuronas.

  10: El número de neuronas en esta capa, que corresponde al número de clases en el conjunto de datos CIFAR-10.

- ***activation='softmax':*** La función de activación softmax se utiliza para convertir las salidas en probabilidades. Cada neurona de esta capa representa una clase y la activación softmax asegura que las salidas sumen 1, permitiendo interpretar las salidas como probabilidades.

6. Compilación del modelo

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

- Compila el modelo especificando el optimizador Adam, la función de pérdida categorical_crossentropy para clasificación multiclase y la métrica accuracy para evaluar el rendimiento.

7. Entrenamiento del modelo

En esta línea del código se entrena el modelo usando los datos de entrenamiento (x_train, y_train), durante 10 épocas y con un tamaño de lote de 64. Usa los datos de prueba (x_test, y_test) para validar el modelo durante el entrenamiento. El historial del entrenamiento se guarda en la variable history.

```
history = model.fit(x_train, y_train, epochs=10, batch_size=64, validation_data=(x_test, y_test))
```

```
Epoch 1/10
782/782 [==============================] - 62s 78ms/step - loss: 1.5846 - accuracy: 0.4210 - val_loss: 1.3434 - val_accuracy: 0.5060
Epoch 2/10
782/782 [==============================] - 65s 83ms/step - loss: 1.2424 - accuracy: 0.5555 - val_loss: 1.1711 - val_accuracy: 0.5784
Epoch 3/10
782/782 [==============================] - 62s 79ms/step - loss: 1.0954 - accuracy: 0.6150 - val_loss: 1.0296 - val_accuracy: 0.6378
Epoch 4/10
782/782 [==============================] - 66s 85ms/step - loss: 0.9883 - accuracy: 0.6533 - val_loss: 0.9763 - val_accuracy: 0.6621
Epoch 5/10
782/782 [==============================] - 63s 80ms/step - loss: 0.9153 - accuracy: 0.6795 - val_loss: 0.9620 - val_accuracy: 0.6692
Epoch 6/10
782/782 [==============================] - 66s 85ms/step - loss: 0.8563 - accuracy: 0.7008 - val_loss: 0.9206 - val_accuracy: 0.6770
Epoch 7/10
782/782 [==============================] - 65s 83ms/step - loss: 0.8077 - accuracy: 0.7184 - val_loss: 0.9147 - val_accuracy: 0.6865
Epoch 8/10
782/782 [==============================] - 66s 84ms/step - loss: 0.7658 - accuracy: 0.7326 - val_loss: 0.8681 - val_accuracy: 0.7052
Epoch 9/10
782/782 [==============================] - 64s 81ms/step - loss: 0.7325 - accuracy: 0.7443 - val_loss: 0.8588 - val_accuracy: 0.7098
Epoch 10/10
782/782 [==============================] - 65s 83ms/step - loss: 0.6966 - accuracy: 0.7554 - val_loss: 0.8662 - val_accuracy: 0.7059
```

a) model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

- ***model.compile():*** Esta es una función del modelo en Keras que se utiliza para compilar el modelo después de haber sido configurado. Cuando se compila un modelo, se especifica la función de pérdida, el optimizador y las métricas que se utilizarán durante el entrenamiento y la evaluación del modelo.

- ***optimizer='adam':*** Este parámetro especifica el optimizador que se utilizará durante el entrenamiento del modelo. En este caso, se utiliza el optimizador Adam. Adam es un algoritmo de optimización popular y eficiente que se utiliza comúnmente en el entrenamiento de redes neuronales. Ajusta automáticamente la tasa de aprendizaje durante el entrenamiento y suele ser una buena opción para una variedad de problemas de aprendizaje profundo.

- ***loss='categorical_crossentropy':*** Este parámetro especifica la función de pérdida que se utilizará durante el entrenamiento del modelo. En el caso de un problema de clasificación multiclase (como el que estamos abordando aquí, con el conjunto de datos CIFAR-10), se utiliza la entropía cruzada categórica como función de pérdida. Esta función de pérdida compara la distribución de probabilidad predicha por el modelo con la distribución de probabilidad real de las etiquetas de clase. La entropía cruzada categórica es una medida de la discrepancia entre estas distribuciones y es una opción común para problemas de clasificación multiclase.

- ***metrics=['accuracy']:*** Este parámetro especifica las métricas que se utilizarán para evaluar el rendimiento del modelo durante el entrenamiento y la

evaluación. En este caso, se utiliza la métrica de precisión (accuracy), que mide la fracción de imágenes correctamente clasificadas. Es una métrica comúnmente utilizada para problemas de clasificación y proporciona una medida intuitiva del rendimiento del modelo en términos de la precisión de las predicciones.

En resumen, esta línea de código configura y prepara el modelo para el entrenamiento utilizando el optimizador Adam, la función de pérdida de entropía cruzada categórica y la métrica de precisión. Estas opciones son adecuadas para un problema de clasificación multiclase como el que estamos abordando aquí con el conjunto de datos CIFAR-10.

8. Evaluación del modelo

Evalúa el modelo usando los datos de prueba y guarda la pérdida (test_loss) y la precisión (test_acc). Imprime la precisión del modelo en los datos de prueba. Se realiza esta evaluación para comprender qué tan bien el modelo se desempeña en datos que no ha visto durante el entrenamiento. Es importante evaluar el modelo en un conjunto de datos de prueba separado para verificar si está generalizando correctamente, es decir, si puede realizar predicciones precisas en datos que no ha visto durante el entrenamiento. Esto ayuda a detectar el sobreajuste (overfitting) o la subestimación (underfitting) del modelo y ajustar su rendimiento en consecuencia.

```
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test accuracy: {test_acc}")
```

```
313/313 [==============================] - 5s 16ms/step - loss: 0.8662 - accuracy: 0.7059
Test accuracy: 0.7059000134468079
```

a) model.evaluate(x_test, y_test)

Esta función evalúa el modelo utilizando el conjunto de datos de prueba (`x_test`) y sus etiquetas correspondientes (`y_test`). Toma como entrada las características de prueba y las etiquetas de las clases verdaderas y las utiliza para calcular la pérdida y las métricas especificadas durante la compilación del modelo.

b) test_loss, test_acc = ...

La función `evaluate` devuelve dos valores: la pérdida obtenida en el conjunto de datos de prueba (`test_loss`) y la precisión de la clasificación (`test_acc`). Estos valores se asignan a las variables `test_loss` y `test_acc` respectivamente.

c) **print(f"Test accuracy: {test_acc}")**

Finalmente, se imprime la precisión del modelo en el conjunto de datos de prueba. Utilizando un formato de cadena f, se muestra el valor de `test_acc` (precisión) como parte del mensaje. Esto proporciona una forma clara de comprender qué tan bien el modelo generaliza sobre datos no vistos.

9. ***Graficar las pérdidas (loss) durante el entrenamiento***

```
if history:
    plt.plot(history.history['loss'], label='Pérdida de entrenamiento')
    plt.plot(history.history['val_loss'], label='Pérdida de validación')
    plt.xlabel('Épocas')
    plt.ylabel('Pérdida')
    plt.legend()
    plt.title('Pérdida durante el entrenamiento y validación')
    plt.show()
else:
    print("No se pudo obtener el historial del entrenamiento.")
```

- ***if history:*** Esta línea verifica si el objeto history no es None o si existe. El objeto history es el resultado del método fit de un modelo de Keras, el cual contiene información sobre el entrenamiento del modelo.

- ***plt.plot(history.history['loss'], label='Pérdida de entrenamiento'):*** Esta línea grafica los valores de pérdida de entrenamiento almacenados en history.history['loss']. label='Pérdida de entrenamiento' establece una etiqueta para la leyenda del gráfico.

- ***plt.plot(history.history['val_loss'], label='Pérdida de validación'):*** Esta línea grafica los valores de pérdida de validación almacenados en history.history['val_loss']. label='Pérdida de validación' establece una etiqueta para la leyenda del gráfico.

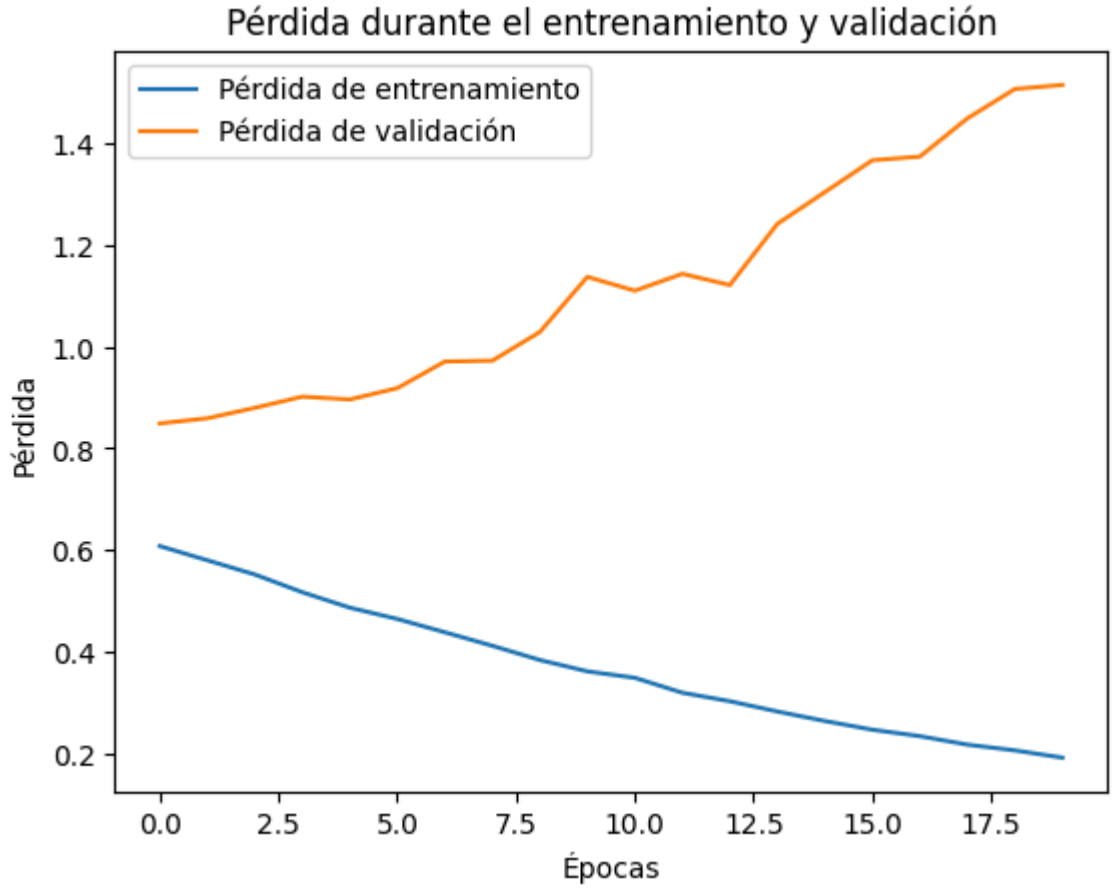
- ***plt.xlabel('Épocas'):*** Establece la etiqueta del eje X como "Épocas". Las épocas son iteraciones completas sobre el conjunto de datos de entrenamiento durante el proceso de entrenamiento del modelo.

- ***plt.ylabel('Pérdida'):*** Establece la etiqueta del eje Y como "Pérdida". La pérdida es una métrica que indica qué tan mal está prediciendo el modelo; un valor más bajo indica un mejor rendimiento.

- ***plt.legend():*** Añade una leyenda al gráfico. La leyenda utiliza las etiquetas proporcionadas en las llamadas a plt.plot().

- ***plt.title('Pérdida durante el entrenamiento y validación'):*** Establece el título del gráfico como "Pérdida durante el entrenamiento y validación".

- ***plt.show():*** Muestra el gráfico en pantalla.
- ***else: print("No se pudo obtener el historial del entrenamiento."):*** Si el objeto history es None o no existe, se ejecuta este bloque, imprimiendo un mensaje que indica que no se pudo obtener el historial del entrenamiento.



Explicación del gráfico

Las curvas de pérdida de entrenamiento y validación no se cruzan nunca lo cual indica que el modelo está aprendiendo de manera efectiva y generalizando bien.

10. Función para predecir una nueva imagen

Esta función carga una imagen, la prepara para la entrada del modelo, realiza una predicción utilizando el modelo previamente entrenado y devuelve la clase predicha. Las acciones realizadas, como la redimensión, expansión de dimensiones y normalización, son necesarias para asegurar que la imagen sea compatible con el modelo y produzca resultados precisos.

```python
def predict_image(image_path):
    img = load_img(image_path, target_size=(32, 32))
    img_array = img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array = img_array.astype('float32') / 255.0
    prediction = model.predict(img_array)
    class_index = np.argmax(prediction)
    class_names = ['avión', 'automóvil', 'pájaro', 'gato', 'ciervo', 'perro', 'Rana', 'caballo', 'barco', 'camión']
    return class_names[class_index]
```

- ***def predict_image(image_path):*** Define una función llamada predict_image que toma un parámetro image_path, que es la ruta a la imagen que se desea predecir.

- ***img = load_img(image_path, target_size=(32, 32)):*** Carga la imagen desde la ruta especificada (image_path) y la redimensiona a 32x32 píxeles, que es el tamaño requerido por el modelo de predicción.

- ***img_array = img_to_array(img):*** Convierte la imagen cargada (img) en un array de NumPy. Este array tiene la forma (32, 32, 3), donde 3 representa los canales de color RGB.

- ***img_array = np.expand_dims(img_array, axis=0):*** Añade una dimensión extra al array de la imagen para que tenga la forma (1, 32, 32, 3). Los modelos de Keras esperan un lote de imágenes como entrada, por lo que necesitamos esta dimensión adicional.

- ***img_array = img_array.astype('float32') / 255.0:*** Convierte los valores de los píxeles de enteros a flotantes y los normaliza dividiendo por 255.0, de modo que los valores estén en el rango [0, 1]. Esto es importante para que el modelo funcione correctamente, ya que fue entrenado con datos normalizados.

- ***prediction = model.predict(img_array):*** Utiliza el modelo entrenado para predecir la clase de la imagen. El resultado prediction es un array que contiene las probabilidades predichas para cada clase.

- ***class_index = np.argmax(prediction):*** Encuentra el índice de la clase con la probabilidad más alta. np.argmax devuelve el índice del valor máximo en el array prediction.

- ***class_names = ['avión', 'automóvil', 'pájaro', 'gato', 'ciervo', 'perro', 'Rana', 'caballo', 'barco', 'camión']:*** Define una lista con los nombres de las clases correspondientes a los índices de salida del modelo. Estas son las etiquetas de las 10 clases del conjunto de datos CIFAR-10.

- ***return class_names[class_index]:*** Devuelve el nombre de la clase correspondiente al índice class_index. Esta es la predicción final del modelo, indicando a cuál de las 10 clases pertenece la imagen.

11. Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

- Monta Google Drive en el entorno de Colab para acceder a archivos e imágenes almacenados allí.

12. Prueba con una imagen nueva

Este bloque de código se utiliza para probar el modelo entrenado con varias imágenes nuevas y verificar su capacidad para clasificar correctamente las imágenes que no se han utilizado durante el entrenamiento. Esto es importante para evaluar la capacidad de generalización del modelo.

```
[29] #Pruebas
     # Prueba con una imagen nueva 01
     image_path = '/content/drive/MyDrive/Ramos - Torres: Evaluacion parcial Tecnologias emergentes/caballo.jpg'
     predicted_class = predict_image(image_path)
     print(f"La clase prevista de la imagen 1 es: {predicted_class}")
     # Prueba con una imueagen nueva 02
     image_path1 = '/content/drive/MyDrive/Ramos - Torres: Evaluacion parcial Tecnologias emergentes/rana.jpg'
     predicted_class1 = predict_image(image_path1)
     print(f"La clase prevista de la imagen 2 es: {predicted_class1}")

     # Prueba con una imagen nueva 03
     image_path2 = '/content/drive/MyDrive/Ramos - Torres: Evaluacion parcial Tecnologias emergentes/carro.jpg'
     predicted_class2 = predict_image(image_path2)
     print(f"The predicted class for image 3 is: {predicted_class2}")

     # Prueba con una imagen nueva 04
     image_path3 = '/content/drive/MyDrive/Ramos - Torres: Evaluacion parcial Tecnologias emergentes/perro.jpg'
     predicted_class3 = predict_image(image_path3)
     print(f"La clase prevista de la imagen 4 es: {predicted_class3}")

     # Prueba con una imagen nueva 05
     image_path4 = '/content/drive/MyDrive/Ramos - Torres: Evaluacion parcial Tecnologias emergentes/avion.jpg'
     predicted_class4 = predict_image(image_path4)
     print(f"La clase prevista de la imagen 5 es: {predicted_class4}")
```

```
1/1 [==============================] - 0s 23ms/step
La clase prevista de la imagen 1 es: caballo
1/1 [==============================] - 0s 25ms/step
La clase prevista de la imagen 2 es: Rana
1/1 [==============================] - 0s 29ms/step
The predicted class for image 3 is: automóvil
1/1 [==============================] - 0s 22ms/step
La clase prevista de la imagen 4 es: perro
1/1 [==============================] - 0s 22ms/step
La clase prevista de la imagen 5 es: avión
```

- ***image_path:*** Define la ruta del archivo de imagen que se utilizará para la predicción. En este caso, la imagen se encuentra en Google Drive y la ruta es especificada. Se debe reemplazar esta ruta con la ruta de la imagen que se desea utilizar.

- ***predicted_class = predict_image(image_path):*** Llama a la función predict_image (definida anteriormente) con la ruta de la imagen como argumento. Esta función carga

la imagen, la preprocesa y utiliza el modelo para predecir su clase. La clase predicha se almacena en la variable predicted_class.

- ***print(f"La clase prevista es: {predicted_class}"):*** Imprime en la consola la clase predicha por el modelo. La cadena de formato f"La clase prevista es: {predicted_class}" permite insertar el valor de predicted_class en la cadena de texto.